آموزش صفر تا صد — نسخه ۲۰۲۶

Command Line (CLI)
از صفر مطلق تا قدرت کامل

از اولین Enter تا اسکریپت‌های شخصی و کنترل سرور

PATH & Processes | grep & pipes | Bash Scripting | Git Bash | SSH & curl

۱۳ فصل · تمرین با جواب · ده ارور کلاسیک · چیت‌شیت و گلاساری

فهرست مطالب

1	شروع دوره	نقشه دوره و زمین تمرین
2	فصل ۱ — CLI چیست و چرا	terminal vs shell، چرا CLI، الگوی دستورها
3	فصل ۲ — آماده‌سازی ترمینال	VS Code، Git Bash، کلیدهای مقدس (Tab/Ctrl+R)
4	فصل ۳ — ناوبری	pwd/ls/cd، مسیر مطلق/نسبی، ~ و ..
5	فصل ۴ — جعبه‌ابزار فایل‌ها	mkdir/cp/mv/rm، خواندن فایل‌ها، wildcard
6	فصل ۵ — راهنما گرفتن	--help، which، alias، ~/.bashrc
7	فصل ۶ — Pipes و Redirection		و > و >>، stdin/stdout/stderr
8	فصل ۷ — grep و find	جستجوی متن و فایل‌ها، سناریوهای واقعی
9	فصل ۸ — دسترسی‌ها و اولین اسکریپت	rwx، chmod، sudo، hello.sh
10	فصل ۹ — متغیرهای محیطی و PATH	export، راز .env، command not found
11	فصل ۱۰ — پروسه‌ها و پورت‌ها	پس‌زمینه، kill، آزاد کردن پورت ۳۰۰۰
12	فصل ۱۱ — جعبه‌ابزار دولوپر	node/npm/npx، curl، ssh، tar
13	فصل ۱۲ — اسکریپت‌نویسی Bash	آرگومان، شرط، حلقه، newproj.sh
14	فصل ۱۳ — چیت‌شیت و گلاساری	مرور سریع + واژه‌نامه + اورهای کلاسیک

🎓 دوره جامع Command Line (CLI) — از «ترس از سیاه‌صفحه» تا قدرت کامل

> 🤖 **تولید شده توسط هوش مصنوعی:** این دوره به‌طور کامل توسط مدل **GLM-5.3-Flash** تولید و تدوین شده است. **نسخه دوره:** ۲۰۲۶ — با Git Bash روی ویندوز + تفاوت‌های PowerShell/cmd هرجا لازم شد **پیش‌نیاز:** هیچ! حتی اگر تا حالا یک دستور هم دستی اجرا نکرده باشید، این دوره از صفر مطلق تا سطح پیشرفته شما را همراهی می‌کند.

🎯 چرا یادگیری خط فرمان (CLI) حیاتی است؟

در دنیای مدرن نرم‌افزار، ابزارهای اصلی توسعه مانند Git، npm/pnpm، Node.js، پایگاه‌های داده، ابزارهای ابری و SSH **همگی مبتنی بر خط فرمان هستند.** تکیه بر رابط‌های گرافیکی محدودیت‌های زیادی ایجاد می‌کند، در حالی که خط فرمان سرعت، دقت و امکان اتوماسیون بی‌نظیری را فراهم می‌سازد. خبر خوب: یادگیری CLI بسیار لذت‌بخش و منطقی است، زیرا نتایج آن فوری و در کار روزمره کاملاً ملموس است.

پس از اتمام این دوره: - هر پروژه یا مخزنی را به سرعت دریافت و با پکیج منیجرها اجرا می‌کنید. - خطاهای مسدود شدن پورت‌ها یا کرش پروسه‌ها را در چند ثانیه شناسایی و برطرف می‌سازید. - لاگ‌های سنگین را با `grep` فیلتر کرده و فایل‌ها را با یک دستور مدیریت می‌کنید. - اسکریپت‌های اتوماسیون اختصاصی خود را برای حذف کارهای تکراری می‌نویسید.

🗺️ نقشه دوره

فصل	موضوع	چی یاد می‌گیری
فصل ۱	CLI چیست و چرا؟	terminal vs shell، انواع شل، چرا دولوپرها عاشقش‌ان
فصل ۲	ترمینال تو	Windows Terminal، Git Bash، ترمینال VS Code، آناتومی prompt
فصل ۳	ناوبری	cd، ls، pwd، مسیر مطلق/نسبی، Tab، تاریخچه
فصل ۴	فایل‌ها و پوشه‌ها	rm، mv، cp، touch، mkdir، خواندن فایل‌ها، wildcard ها
فصل ۵	راهنما گرفتن	--help، man، which، alias — «حافظه‌ی بیرونی» تو
فصل ۶	Pipes و Redirection ⭐	| و > و >> — ابرقدرت واقعی شل
فصل ۷	جستجو	grep و find — غول‌چراغ پروژه‌های واقعی
فصل ۸	دسترسی‌ها و اولین اسکریپت	chmod، sudo، shebang، فایل اجرایی خودت
فصل ۹	متغیرهای محیطی و PATH	export، رازِ «command not found»، .env
فصل ۱۰	پروسه‌ها	پس‌زمینه، Ctrl+C، kill، **آزاد کردن پورت ۳۰۰۰!**
فصل ۱۱	جعبه‌ابزار دولوپر	tar، ssh، curl، node/npm/npx — ابزارهایی که هر روز می‌بینی
فصل ۱۲	اسکریپت‌نویسی Bash	متغیر، شرط، حلقه — اتوماسیون کارهای تکراری
فصل ۱۳	چیت‌شیت و گلاساری	مرور سریع + مسیر ادامه (PowerShell و...)

🧪 زمین تمرین

تمام تمرین‌ها در یک پوشه امن انجام می‌شن — خرابکاریش مجانیه:

```
mkdir ~/cli-playground && cd ~/cli-playground
```

(اگه این دو خط رو الان نفهمیدی — دقیقاً همون جاییه که فصل ۳ شروع می‌شه 😄)

قانون طلایی دوره: ترمینال با Enter کارت رو *اجرا* می‌کنه، نه با تایپ. تا Enter نزدی هیچی اتفاق نمی‌افته — پس با خیال راحت تایپ کن. و هر چیزی که نمی‌فهمی، Tab بزن (فصل ۳) و --help بگیر (فصل ۵) — این دو تا، حافظه‌ت رو لازم ندارن.

📋 پیش‌نیازهای فنی

ابزار	وضعیت	چک کردن
Git for Windows (شامل Git Bash)	الزامی (پایگاه محیط Bash در ویندوز)	`git --version` در Git Bash
VS Code	توصیه‌شده	ترمینال داخلی برای گردش کار سریع
Windows Terminal	اختیاری ولی توصیه‌شده	تجربه مدرن تب‌بندی ترمینال‌ها (فصل ۲)

> 💡 **کدوم شل؟** کل دوره با **Bash** (داخل Git Bash) — چون استاندارد جهانیِ سرورها و مک‌هاست و چیزی که یاد بگیری همه‌جا یکسانه. معادل‌های PowerShell/cmd رو کجاها لازم شد جدا ذکر می‌کنم.

📄 تبدیل به PDF

نسخه PDF رنگی راست‌به‌چپ این دوره (فونت وزیرمتن) در همین پوشه آماده است: **«دوره کامل Command Line.pdf»**.

برای بازسازی بعد از ویرایش: `python build/build_pdf.py` + رندر با Chrome headless (دستور دقیق در پرامپت آماده‌ی پوشه والد).

🏷️ کلمات کلیدی و برچسب‌های سئو (SEO Keywords & Tags)

تگ‌های پیشنهادی برای مخزن گیت‌هاب (Topics):

```
cli, command-line, bash, terminal, powershell, آموزش-ترمینال, آموزش-خط-فرمان, git-bash, dev-tools, persian-tutorial, farsi
```

کلمات کلیدی جستجو:

آموزش خط فرمان و ترمینال به زبان فارسی, کتاب یادگیری دستورات CLI pdf, آموزش ترمینال برای برنامه نویسان, دستورات پرکاربرد لینوکس و ویندوز در ترمینال

📥 دانلود مستقیم نسخه چاپی و PDF کتاب

برای دسترسی و دانلود مستقیم فایل PDF کامل این دوره آموزشی، به بخش **Releases** همین ریپازیتوری مراجعه کنید یا فایل PDF قرار داده شده در ریشه مخزن را دریافت نمایید.

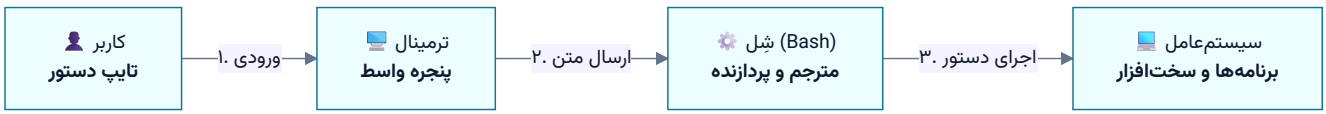
فصل ۱ — CLI چیست و چرا دولوپرها عاشقش‌ان؟

🎯 **هدف این فصل:** مفهوم‌های بنیادی رو مرتب کنی: ترمینال چیست، شل چیست، فرقشون چیه، و چرا صنعت نرم‌افزار هنوز بعد از ۵۰ سال به این «سیاه‌صفحه» وابسته‌ست. هنوز هیچ دستوری نمی‌زنیم — فقط ذهنیت.

۱.۱ — سه کلمه‌ای که قاطی می‌شن: Console، Terminal، Shell

واژه	چیه	شبیه به
Shell	برنامه‌ای که دستورات متنی رو *می‌فهمه و اجرا می‌کنه* (Bash، PowerShell، Zsh)	مغز — مترجم تو با سیستم‌عامل
Terminal	پنجره‌ای که شل رو *نگه می‌داره و نمایش می‌ده* (Windows Terminal، iTerm، ترمینال VS Code)	بدن — پنجره و صفحه‌کلید
Console	تاریخچه‌ای: صفحه‌کلید+نمایشگر فیزیکی رایانه‌های قدیمی	امروزه معمولاً = terminal

رابطه‌شون:



🔁 **مسیر بازگشت خروجی:** سیستم‌عامل نتیجه پردازش را به **شِل** برمی‌گرداند ← شِل متن خروجی را به **ترمینال** می‌سپارد ← ترمینال آن را روی صفحه به **کاربر** نمایش می‌دهد.

پس وقتی می‌گیم «ترمینال باز کن»، عملاً یعنی «یک پنجره باز کن که داخلش Bash اجرا شده باشه».

شل‌های اصلی که خواهی دید

شل	کجا	نکته
Bash	لینوکس، مک، Git Bash ویندوز	⭐ استاندارد جهانی — محور این دوره
Zsh	مک (پیش‌فرض جدید) + oh-my-zsh	تقریباً سازگار با Bash
PowerShell	ویندوز	سبک متفاوت: cmdlet ها مثل `Get-ChildItem`
cmd	ویندوز قدیمی	`dir` به جای `ls` — میراث DOS

> 💡 **چرا Bash یاد بگیریم و نه PowerShell؟** چون هر سرور لینوکسی روی زمین، هر آموزش گیت/داکر/Node و هر شرکت لینوکس‌محور با Bash صحبت می‌کنه. PowerShell فقط در محیط ویندوز-محوره. Bash یاد بگیری، همه‌جا مسلطی؛ cmd یاد بگیری، فقط تو یه گوشه‌ی ویندوز کارت راهه.

۱.۲ — چرا رابط متنی، وقتی گرافیکی داریم؟

سوال منصفانه‌ست. جواب در سه کلمه: **قابلیت ترکیب، خودکارسازی، سرعت.**

۱) قابلیت ترکیب (Composition): در GUI هر ابزار جزیره‌ایه. در CLI همه ابزارها با یک استاندارد مشترک (متن) حرف می‌زنن — پس زنجیر می‌شن:

```
cat error.log | grep "ERROR" | sort | uniq -c | sort -rn
# بخون لاگ رو، فقط خطاهای ERROR رو بردار، مرتب کن، بشمار، پربسامدها اول
```

یک خط، کار یک ابزار تحلیلی کرد. در GUI برای این کار چند نرم‌افزار و کلی کپی/پیست لازم بود.

۲) خودکارسازی: هر چیزی که دستی تایپ می‌شه، می‌شه توی یک فایل اسکریپت گذاشت و هزار بار اجراش کرد. دکمه‌ی GUI رو نمی‌تونی «ذخیره و تکرار» کنی؛ دستور CLI رو می‌تونی.

۳) سرعت و دقت: برای کارهای تکراری (ساخت پروژه، پیمایش بین دوتا پوشه دور، نصب ابزار) تایپ چند حرف از سوییچ‌دار بودن دست‌هات سریع‌تر از حرکت موسه.

۴) و مهم‌ترین برای تو — کل اکوسیستم دولوپری CLI-محوره: `git` ، `npm` ، `npx create-next-app` ، `psql` ، `ssh` ، `docker` ، `gh` — همه فقط کامند لاین دارن (یا GUI هاشون ناقصه). دولوپر بدون CLI مثل آشپز بدون چاقوئه.

🔑 **چشم‌انداز دوره:** قرار نیست «حافظه‌ت رو با صدها دستور پر کنی». قراره *روش فکر کردن* به CLI رو یاد بگیری: هر ابزار یک کار کوچیک رو عالی انجام می‌ده، و تو اونها رو مثل لگو کنار هم می‌چینی. حافظه رو به Tab و --help می‌سپاریم (فصل ۳ و ۵).

۱.۳ — یک نگاه به آینده: شکل یک دستور

تمام دستوراتی که یاد می‌گیری یک الگوی مشترک دارن — حالا که می‌بینی، بقیه دوره فقط پر کردن جدوله:

```
command  -short-flag  --long-flag  argument  argument
   │          │            │          │
   │          │            │          └──.(.  مسیر  فایل، (اسم شه اجرا «چه چیزی» روی
   │          │            └── (کامل و خوانا) بلند گزینه
   │          └── (حرف‌خلاصه) کوتاه گزینه
   └── برنامه اسم
```

مثال واقعی که تا حالا دیدی:

```
git commit -m "برنامه #      "پیام=git، فلگ=-m، پیام"=آرگومان"
npx create-next-app my-app --typescript
```

و دو قاعده جهانی که از همین الان بدونی:

1. **حساس به بزرگی/کوچکی حروفه** — `ls` کار می‌کنه، `LS` نه
2. **فلگ‌ها اختیاری‌ان، ترتیب آرگومان‌ها مهمه** — `git -m commit` غلطه

۱.۴ — افسانه‌های ترسناک (و حقیقتشون)

افسانه 😱	حقیقت ✅
«اگه اشتباه بزنم سیستم می‌سوزه»	شل دستور ناقص رو اصلاً اجرا نمی‌کنه؛ فقط ارور می‌ده. حتی حذف فایل‌ها هم محدوده به همون مسیری که درشی
«باید صدها دستور حفظ باشم»	۲۰-۳۰ دستور ۹۵٪ کار روزمره‌ست؛ بقیه با --help
«ترمینال برای هکرهاست»	ترمینال برای *هر کسی که کارش خودکارسازیه* — یعنی هر دولوپر
«ویندوز CLI نداره»	Git Bash یک Bash واقعی رو روی ویندوزت داری؛ و Windows Terminal همه رو یک‌جا جمع می‌کنه

⚠️ فقط یک استثنای واقعی برای «ترس»: دستورهای تخریبی با مسیر اشتباه (`rm -rf` روی مسیر غلط) خطرناکن — مثل هر ابزار قدرتمند دیگه. قانون‌های ایمنی رو در فصل ۴ می‌سازیم و بعد دیگه نگرانی معنا نداره.

۱.۵ — جایگاه CLI در اکوسیستم و نقشه راه برنامه‌نویسی

ابزارهای استاندارد دنیای مهندسی نرم‌افزار و کاربرد هر یک در سرفصل‌های این دوره:

ابزار	کاربرد در پروژه‌های واقعی	فصل مربوط
`npx` / `git`	مدیریت نسخه و اجرای پکیج‌های توسعه	۱۱
`psql`	ارتباط و کوئری‌زدن به دیتابیس در ترمینال	۶ (ورودی/خروجی)
`ssh-keygen`	تولید کلیدهای امنیتی برای سرورها و مخازن کد	۸، ۱۱
`npm run dev`	اجرای سرورهای توسعه و پروسه‌های وب	۱۰ (پروسه‌ها)
`.env`	مدیریت متغیرهای محیطی و کلیدهای پیکربندی	۹

خط فرمان در واقع زبان مشترک و زیرساخت ارتباطی میان تمام این ابزارهاست و تسلط بر آن موجب درک عمیق‌تر سازوکار نرم‌افزارها می‌شود.

✅ جمع‌بندی فصل

- **Shell** = مترجم دستورات (Bash ⭐)، **Terminal** = پنجره‌ی نگه‌دارنده‌اش
- CLI سه مزیت دارد: ترکیب‌پذیری (پایپ‌ها)، خودکارسازی (اسکریپت)، سرعت
- الگوی همه دستورات: `command -flag --long-flag argument`
- حساس به حروف؛ فلگ اختیاری؛ ترتیب آرگومان مهم
- ترس‌ها افسانه‌ان؛ فقط تخریب با مسیر غلط واقعیه ← قانون ایمنی در فصل ۴
- Bash یاد بگیری = همه‌جا (لینوکس، مک، ویندوز، سرورها) مسلطی

📝 تمرین فصل ۱

1. با جمله خودت فرق shell و terminal رو بگو (یک جمله کافیه).
2. سه دستور رایج ابزارهای توسعه (مانند git یا npm) را با الگوی `command + flag + argument` باز کن. (مثال: `npm install -g pnpm` ← برنامه npm، فلگ -g، آرگومان install و pnpm)

3. سوال فکری: چرا `git -m commit` غلطه ولی `git commit -m "x"` درسته؟ (راهنمایی: چیه git از مال خودشه؟)

▼ **جواب‌ها**

1. شل برنامه‌ای است که دستور را تفسیر و اجرا می‌کند؛ ترمینال پنجره‌ای است که شل داخلش اجرا و نمایش داده می‌شود.
2. چون ترتیب آرگومان‌ها مهم است: اول برنامه (git)، بعد زیردستور (commit)، بعد فلگ‌ها و آرگومان‌های خودِ آن زیردستور. زیردستور کامیت است که با -m کار می‌کند، نه خود git.

➡️ **فصل بعد:** ترمینالت رو مثل حرفه‌ای‌ها آماده می‌کنیم و اولین Enter رو می‌زنیم!

فصل ۲ — ترمینالت رو آماده کن: اولین Enter

🎯 **هدف:** سه راه ورود به Bash رو بشناسی، ترمینال VS Code رو خونه‌ی اصلیت کنی، و آناتومی خط دستور رو بفهمی. آخر این فصل چند دستور زدی و دستت گرم شده.

۲.۱ — سه راه باز کردن Bash روی ویندوز

راه	چطوری	کی
Git Bash	Start Menu → Git Bash	شروع سریع
ترمینال VS Code ⭐	VS Code باز → `Ctrl+`` (کلید زیر Esc)	خونه اصلی دولوپرها — کنار کدت
Windows Terminal	از Microsoft Store نصب کن؛ تب‌ها داره	اگه ترمینال‌های زیاد داری

نکته‌ی مهم VS Code: به صورت پیش‌فرض ممکنه PowerShell رو باز کنه. بذار Bash رو پیش‌فرضش کنیم:

1. `Ctrl+Shift+P` → تایپ کن: `Terminal: Select Default Profile`
2. **Git Bash** رو انتخاب کن
3. ترمینال‌های باز رو ببند (سطل آشغال بالای پنل) و دوباره `Ctrl+`` — حالا همیشه Bash داری

💡 **و برای پروژه‌ها:** در VS Code روی پوشه پروژه راست‌کلیک → «Open in Integrated Terminal» — ترمینال مستقیم توی همون مسیر پروژه باز می‌شه (درباره «مسیر» فصل ۳).

۲.۲ — آناتومی خط دستور (prompt)

وقتی Git Bash باز می‌شه، چیزی شبیه این می‌بینی:

```
developer@laptop MINGW64 ~/projects
$ 
```

قسمت‌ها رو بشناس:

قسمت	معنی
`developer@laptop`	کاربر@نام‌کامپیوتر
`MINGW64`	اسم محیط Git Bash (مهم نیست حفظش)
`~/projects`	مسیر فعلی (فصل ۳ کامل)
`$`	**prompt** — یعنی «منتظر دستورتم»
`▮`	کرسر — جایی که تایپ می‌شه

🔑 **قاعده طلایی خواندن آموزش‌ها (و همین دوره): در همه‌ی مستندات دنیا، کاراکتر `$` اول خط یعنی «این را در ترمینال تایپ کن» — خودِ `$` را تایپ نکن!** فقط بقیه‌ی خط. (وقتی `$` نیست و `#` است یعنی دستور ادمینی — فعلاً مال ما نیست.)

۲.۳ — سه دستور اول عمرت

Enter رو آماده کن. این سه تا «سلام دنیای» CLI هستن:

```
$ echo "سلام دنیا"
سلام دنیا

$ whoami
developer

$ date
Sat Sep 12 14:32:07 2026
```

- `echo` = هر چی بهش بدی، برمی‌گردونه (مثل `console.log` شل!)
- `whoami` = من کی‌ام (کاربر فعلی)
- `date` = تاریخ و ساعت سیستم

یک نکته‌ی مهم که چشم حرفه‌ای‌ها رو می‌سازه: **دستور در خط prompt اجرا می‌شه و خروجی زیرش میاد، بعد دوباره prompt جدید.** ترمینال یک گفتگوی چرخشی است: تو می‌گی → سیستم جواب می‌ده → تو می‌گی...

۲.۴ — کلیدهای جادویی (قبل از هر دستور دیگه‌ای اینها رو بلد باش)

کلید	کار	ارزش
`Tab`	**تکمیل خودکار** اسم فایل/پوشه/دستور	⭐⭐⭐ هیچوقت اسم کامل ننویس!
`↑` / `↓`	پیمایش تاریخچه دستورات	⭐⭐⭐ دستور قبلی رو دوباره ننویس
`Ctrl+C`	لغو دستور در حال اجرا / خط جدید	⭐⭐⭐ دکمه فرار همیشگی
`Ctrl+L`	پاک کردن صفحه	⭐ تمیزی
`Ctrl+E` / `Ctrl+A`	پرش به ابتدا/انتهای خط	⭐ ویرایش سریع
`Ctrl+R`	جستجو در تاریخچه دستورات	⭐⭐ «اون دستور دیروز چی بود؟»
`Ctrl+U`	پاک کردن از کرسر تا اول خط	اشتباه تایپی طولانی

دو تا از اینها رو الان تمرین کن:

Tab: تایپ کن `ech` بعد Tab بزن — خودش می‌شه `echo` . (کامل‌سازی دستورات هم می‌کنه!)

Ctrl+R: بزن و بعد تایپ کن `echo` — آخرین دستور echo رو از تاریخچه میاره. Enter = اجرا، Tab یا فلش راست = ویرایش.

> 🔑 **فلسفه دوره: حرفه‌ای‌ها کم تایپ می‌کنن. Tab و ↑ و Ctrl+R یعنی «حافظه‌ت رو بیخیال» — سیستم یادته.**

۲.۵ — شخصی‌سازی اولیه (اختیاری ولی لذت‌بخش)

فونت و سایز ترمینال: در VS Code تنظیمات → `terminal.integrated.fontSize` (مثلاً 14). در Git Bash مستقل: راست‌کلیک روی نوار عنوان → Options → Text.

و یک دستور مفید برای همین الان:

```
$ clear          # صفحه رو پاک کن (یا Ctrl+L)
```

✅ جمع‌بندی فصل

- سه راه: Git Bash مستقل، **ترمینال VS Code** (پیش‌فرضش رو Git Bash کن)، Windows Terminal
- آناتومی prompt: کاربر@میزبان + مسیر + `$` — و قانون: `$` **رو تایپ نکن**
- `echo/whoami/date` = اولین دستورات؛ ترمینال گفتگوی چرخشی است

- کلیدهای مقدس: **Tab** (کامل‌سازی)، ↑ (تاریخچه)، **Ctrl+C** (فرار)، **Ctrl+R** (جستجوی تاریخچه)
- حرفه‌ای = کم‌تایپ: حافظه رو به ابزارها واگذار کن

📝 تمرین فصل ۲

1. ترمینال VS Code رو با پیش‌فرض Git Bash تنظیم کن و سه دستور `date` ، `whoami` ، `echo` رو بزن.
2. با Tab: بنویس `who` و Tab بزن — چه پیشنهادهایی می‌ده؟ (`whoami` باید باشه!)
3. تاریخچه: سه دستور مختلف بزن، بعد با `↑` دوتای اول رو برگردون و دوباره اجراشون کن.
4. با Ctrl+R دستور `date` رو از تاریخچه صدا کن.
5. (کشف آزاد) چند بار Tab رو با حروف مختلف (`c` ، `n` ، `g`) امتحان کن — هر چیزی که دیدی یادداشت کن؛ دستورات استاندارد و مهمی مثل `cat` ، `npm` ، `git` و غیره را می‌بینی!

▼ راهنمای تمرین ۵

مثلاً `g` + Tab → شاید `git` و `grep` و `gcc` ...؛ `n` + Tab → `npm` ، `node` ، `nano` ...؛ این لیست، فهرست ابزارهای نصب‌شده‌ی سیستمته — دفعه بعد که اسم یه ابزار رو یادت رفت، همینه که یادت میاره.

➡️ **فصل بعد:** ناوبری — اولین مهارت واقعی: کجام و چطوری برم اونجا؟

فصل ۳ — ناوبری: کجام، چه خبر، چطوری برم اونجا؟

🎯 **هدف:** پرکاربردترین مهارت CLI: جابجایی بین پوشه‌ها با `ls` ، `pwd` و `cd` + مفهوم مسیر مطلق/نسبی که پایه‌ی فهم همه‌ی ابزارهای دولوپریه (از import ها تا git و npm).

۳.۱ — مفهوم کلیدی: Working Directory

شل همیشه توی یک پوشه «نشسته». هر دستوری که می‌زنی نسبت به همون پوشه اجرا می‌شه — بهش می‌گن **Working Directory** (پوشه کاری). مثل اینکه توی اتاقی وایسادی: «فایل profile.js رو باز کن» یعنی مال همین اتاق، مگه اینکه آدرس کامل بدی.

سه ابزار پیمایش:

```
$ pwd           # Print Working Directory — الان کجام؟
/c/Users/developer

$ ls            # لیست محتوای پوشه فعلی
Documents  Downloads  projects

$ cd projects   # Change Directory — برو به پوشه projects
```

⚠️ **تفاوت ظاهری ویندوزی:** Git Bash مسیرها رو با اسلش رو به جلو (/) نشون می‌ده (`/c/Users/...`) — همون استایل لینوکس/مک. در cmd/PowerShell همون مسیر `C:\Users\...` است. دستورات Bash همیشه / می‌خوان.

۳.۲ — `ls` : لیست‌گیر حرفه‌ای

```
$ ls                    # ساده
$ ls -l                 # لیست بلند: دسترسی‌ها، سایز، تاریخ
$ ls -a                 # همه — حتی مخفی‌ها (چیزی که با . شروع می‌شه)
$ ls -lah               # ⭐ ترکیب محبوب: همه + بلند + سایز خوانا
$ ls ~/projects         # لیست یک پوشه دیگه بدون اینکه بری
```

نمونه خروجی `ls -lah` :

```
drwxr-xr-x 1 developer 197121 4.0K Sep 12 10:00 .
drwxr-xr-x 1 developer 197121 4.0K Sep 12 09:41 ..
-rw-r--r-- 1 developer 197121  512 Sep 12 10:02 package.json
drwxr-xr-x 1 developer 197121    0 Sep 12 10:01 src
```

چند چیز از این خروجی (فصل ۸ دسترسی‌ها رو کامل می‌کنه): - خط اول = `d` ، directory خطوط با `-` = فایل - `.` = همین پوشه، `..` = پوشه والد — دوتا اسم مخفی‌ی همیشگی که پایین‌تر عزیز می‌شن - ستون‌های بعدی: کاربر، سایز، تاریخ، اسم

۳.۳ — `cd` و مفهوم مسیر: مطلق در برابر نسبی ⭐

مسیر مطلق: از ریشه شروع می‌شه — آدرس کامل پستی:

```
cd /c/Users/developer/projects/my-app
```

مسیر نسبی: از *جایی که الان هستی* — راهنمای محلی:

```
cd src/components    # برو داخل src، بعد داخل components
```

چهار نماد طلایی مسیر:

نماد	معنی	مثال
`.`	همین پوشه	`./app.js` = فایلی که همین‌جاست
`..`	یک پوشه بالاتر (والد)	`cd ..` = برگرد عقب
`~`	خانه‌ی کاربر (`/c/Users/developer`)	`cd ~` یا `cd ~/projects`
`/`	ریشه + جداکننده مسیر	`/c/Users`

```
$ cd ..              # یک پله بالا
$ cd ../..           # دو پله بالا
$ cd                 # بدون آرگومان: برو خانه (~) — میان‌بر محبوب
$ cd -               # برگرد به پوشه قبلی که توش بودی (toggle!)
```

> 🔑 **این نمادها همه‌جا هستن:** `./` در import های جاوااسکریپت (`import x from './utils'`)، `~/` در تنظیمات SSH و گیت‌هاب، `..` در ساختار URL ها — یک مفهوم، هزار جا.

۳.۴ — Tab: همون کلیدِ فصل ۲، حالا با قدرت کامل

بهترین خبر دوره: Tab *مسیرها* رو هم کامل می‌کنه:

```
$ cd pro<Tab>        # می‌شود: cd projects
$ cd projects/my<Tab>  # می‌شود: cd projects/my-app
```

اگه چند گزینه بود، **دو بار Tab** = نمایش لیست گزینه‌ها. یعنی: هیچوقت نه اسم پوشه رو حفظی، نه غلط تایپی داری.

پیمایش با Tab + فلش‌ها عملاً مثل Explorer می‌شه ولی سریع‌تر.

۳.۵ — تاریخچه و جستجو

```
$ history              # لیست همه دستورات این نشست (شماره‌دار)
$!!                    # دستور قبلی را دوباره اجرا کن
```

- `↑` چند بار = پیمایش عقب
- `Ctrl+R` = جستجوی زنده در تاریخچه (بخشی از دستور رو تایپ کن، Enter بزن)

۳.۶ — چرخه کامل یک ناوبری واقعی

سناریوی واقعی: «می‌خوام پروژه Next.js خودم رو ببینم»:

```
$ cd ~                          # خانه
$ cd projects                   # وارد پوشه پروژه‌ها (با Tab!)
$ ls                            # کدوم پروژه‌ها دارم؟
my-shop  my-blog  test-app
$ cd my-shop                    # ورود به پروژه
$ ls                            # ساختار پروژه: app, src, package.json...
$ pwd                           # تأیید: /c/Users/developer/projects/my-shop
$ code .                        # باز کن کل پروژه را در VS Code! ⭐
```

`code .` نقطه‌ی اتصال دو دنیاست: از دل ترمینال، همون پوشه‌ای که توشی رو با GUI باز می‌کنی. (`.` = همینجا!)

✅ جمع‌بندی فصل

- شل همیشه در یک Working Directory نشسته؛ همه دستورها نسبت به اون
- `pwd` (کجام؟) / `ls -lah` (چه خبر؟) / `cd` (برم اونجا)
- مسیر مطلق از `/` شروع می‌شه، نسبی از جای فعلی؛ نمادها: `/` `~` `..` `.`
- `cd` بدون آرگومان = خانه؛ `cd -` = برگشت
- **Tab** مسیرها رو هم کامل می‌کنه — پیمایش بدون حفظ کردن
- `code .` = باز کردن پروژه فعلی در VS Code

📝 تمرین فصل ۳ (زمین تمرین: `mkdir ~/cli-playground && cd ~/cli-playground`)

1. پیمایش کامل: از خانه شروع کن، با Tab به `cli-playground` برو، `pwd` رو بزن، با `cd ..` برگرد، با `cd -` دوباره برگرد داخلش.

2. ساختار بساز: سه پوشه `web` و `docs` و `scripts` بساز (`mkdir web docs scripts`)، بعد به `web` برو، بعد *بدون رفتن به ریشه* مستقیم به `docs` برو (راهنمایی: `..`).
3. `ls -a` رو در پوشه‌ات بزن — دو تا اسم مخفی همیشگی رو ببین (`.` و `..`).
4. چالش: در یک دستور `cd` از هر جای عمقی، مستقیم به خانه برو (دو راه مختلف پیدا کن!).

▼ **جواب تمرین ۲ و ۴**

```
# تمرین ۲:
mkdir web docs scripts
cd web
cd ../docs        # از web به docs بدون رد شدن از ریشه

# تمرین ۴ — دو راه:
cd                # راه ۱: بدون آرگومان
cd ~              # راه ۲: نماد خانه
```

➡️ **فصل بعد:** حالا که راه می‌ری، وقت ساختن و جابجایی فایل‌هاست — جعبه‌ابزار کامل فایل‌ها.

فصل ۴ — ساختن، جابجایی و حذف: جعبه‌ابزار فایل‌ها

🎯 **هدف:** مدیریت کامل فایل و پوشه با دستور: mkdir، touch، cp، mv، rm + خواندن فایل‌ها + الگوهای wildcard. آخر فصل، Explorer رو عملاً کنار می‌ذاری.

۴.۱ — ساخت: mkdir و touch

```
$ mkdir blog                    # یک پوشه
$ mkdir -p blog/src/components  # ⭐ زنجیره‌ی تو در تو — پدرها را هم بساز
$ touch notes.txt               # فایل خالی بساز (یا تاریخش را تازه کن)
```

(parents) `-p` فلگیه که **همیشه** بهش عادت می‌کنی — بدونش اگه پوشه پدر نباشه ارور می‌گیری، باهاش همه‌ی زنجیره یک‌جا ساخته می‌شه.

۴.۲ — کپی: cp

```
$ cp notes.txt backup.txt          # کپی فایل در همین پوشه
$ cp notes.txt ~/documents/        # کپی به پوشه دیگر
$ cp -r blog blog-backup           # ⭐ کپی پوشه با محتوا (-r = recursive)
```

🔑 **قاعده ذهنی:** برای هر چیزی که *داخلش محتوا داره* (پوشه) نسخه `-r` (recursive) لازمه — این قاعده در `cp` ، `rm` و... یکسانه.

۴.۳ — جابجایی و تغییر نام: mv (دو کار با یک دستور!)

```
$ mv notes.txt old-notes.txt       # تغییر نام (جابجایی به اسم جدید!)
$ mv old-notes.txt ~/documents/    # انتقال به پوشه دیگر
$ mv old-notes.txt ~/documents/archive.txt   # انتقال + تغییر نام همزمان
$ mv *.txt txt-files/              # همه فایل‌های txt را یکجا ببر
```

دقت کن: «تغییر نام» فقط `mv` به اسم جدید در همون پوشه‌ست — چون در فایل‌سیستم، اسم و مسیر یک چیزین.

۴.۴ — حذف: rm (با احترام! ⚠️)

```
$ rm file.txt              # حذف فایل
$ rm -r folder/            # حذف پوشه با محتوا (recursive)
$ rm -rf folder/           # فورس + recursive — بدون هیچ سوال و تأییدی!
```

`rm -rf` معروف‌ترین دستور ترسناک دنیاست. حقیقتش: کاملاً امنه *اگر مسیرش درست باشه* — مثل چاقوی سرآشپز. سه قانون ایمنی:

1. **قبل از rm سنگین، `ls` بزن** — ببین دقیقاً چی می‌خوای حذف کنی (همون مسیر رو با ls تست کن)
2. **از `-f` بلا استفاده استفاده نکن** — معمولاً `rm -r` کافیه و اگه فایل محافظت‌شده‌ای جلوش باشه، خودت تصمیم می‌گیری
3. **Trash نداریم!** حذف با rm = حذف واقعی. (برای چیزهای گرون: اول `cp -r` بکاپ بگیر یا — بهتر — کل پروژه رو توی git داشته باش!)

> 💡 **در PowerShell و cmd معادل‌ها متفاوت‌ان:** `Remove-Item` / `del` و `Remove-Item -Recurse` / `rmdir` **کار می‌کنیم، همون rm برای تو استانداردِ.** `/s`. ولی چون ما Bash

۴.۵ — خواندن فایل‌ها: پنج ابزار، پنج حالت

```
$ cat file.txt          # (برای فایل‌های کوچیک) کل محتوا بریز تو صفحه
$ less file.txt         # ⭐ صفحه‌به‌صفحه — برای فایل‌های بزرگ
                        #   ↑/↓ اسکرول، Space صفحه بعد، / جستجو=کلمه، q خروج
$ head -n 20 file.txt   # ۲۰ خط اول
$ tail -n 20 file.txt   # ۲۰ خط آخر
$ tail -f server.log    # ⭐ (لاگ سرور) دنبال کن زنده! — Ctrl+C برای خروج
```

`tail -f` (follow) ابزار رازهای دولوپرهاست: لاگ سرور رو زنده نشون می‌ده در حالی که داره نوشته می‌شه. وقتی `npm run dev` یا هر سروری لاگ می‌ده، همین رفتار رو داری می‌بینی.

۴.۶ — Wildcard ها: کار گروهی با الگو

الگوهای علامتی (گلوب) که شل *قبل از اجرا* باز می‌شکنه:

الگو	یعنی	مثال
`*`	هر رشته‌ای (هر چند کاراکتر)	`*.log` همه لاگ‌ها
`?`	دقیقاً یک کاراکتر	`file?.txt` ... file1, file2
`[abc]`	یکی از این حروف	`file[12].txt`

```
$ ls *.md               # همه مارکداون‌ها
$ rm *.tmp              # همه tmp ها (⚠️ اول با ls *.tmp ببین چی حذف می‌شه!)
$ cp ~/projects/*.json .    # همه json های پروژه به اینجا
```

🔑 ترفند ایمنی طلایی: **قبل از هر دستور گروهی خطرناک، همون wildcard رو به `ls` بده.** `ls *.tmp` → چی حذف می‌شه؟ → بعد همون رو با `rm` بزن. دو ثانیه، جلوی هزار تا پشیمونی.

۴.۷ — باز کردن با GUI (پل دو دنیا)

```
$ code .              # پوشه فعلی در VS Code
$ code package.json   # یک فایل در VS Code
$ start .             # پوشه فعلی در Explorer ویندوز
```

حرفه‌ای واقعی اینا رو باهم ترکیب می‌کنه: پیمایش و عملیات گروهی با CLI، مشاهده/ویرایش ظریف با VS Code.

۴.۸ — چرخه کامل: ساخت یک پروژه بدون موس

```
$ mkdir -p ~/projects/todo/{src,public,docs}    # {} = ساخت گروهی چندتایی!
$ cd ~/projects/todo
$ touch src/index.js README.md
$ ls -R             # درخت کامل را نشون بده (R = recursive)
```

`mkdir -p a/{x,y,z}` یعنی داخل a سه پوشه x و y و z — brace expansion نامشه و کلی وقت توی سرعتت اثر می‌ذاره.

✅ جمع‌بندی فصل

- `mkdir -p` (زنجیره‌ای) و `mkdir -p a/{b,c}` (گروهی) / `touch`
- `cp` برای پوشه = `cp -r` ؛ `mv` = جابجایی + تغییر نام با هم
- `rm` : قانون‌ها — اول `ls` با همون مسیر، `-f` بلااستفاده، Trash نداریم
- خواندن: `cat` (کوچیک)، `less` (بزرگ)، `head/tail` (برش)، `tail -f` (لاگ زنده)
- Wildcard ها با `ls` اول تست شن؛ `code .` پل به GUI

📝 تمرین فصل ۴ (در cli-playground)

1. یک طرح پروژه بساز: `mkdir -p shop/{src/components,src/pages,public,docs}` و با `ls -R` نتیجه رو ببین.
2. سه فایل بساز: `touch a.txt b.txt c.log` — بعد همه‌ی txt ها رو با یک دستور به پوشه docs انتقال بده (wildcard + mv).

3. با `echo` و `cat` بازی کن: `echo "سلام" > file.txt` (بعداً فصل ۶ کاملش می‌کنه — فعلاً فقط ببین فایل ساخته و پر شد) و با `cat` بخونش.
4. `less` رو تجربه کن: `less ~/somedir/somefile` یا هر فایل متنی بلند — با `/` توش کلمه جستجو کن و با `q` بیا بیرون.
5. (تست ایمنی!) پوشه `shop-backup` با cp -r از `shop` بساز، بعد `shop` رو با rm -r حذف کن، بعد `shop-backup` رو به `shop` تغییر نام بده (mv). این چرخه = بکاپ + حذف + بازیابی با سه دستور.

▼ **جواب تمرین ۵**

```
cp -r shop shop-backup
rm -r shop
mv shop-backup shop
```

➡️ **فصل بعد:** دیگه قرار نیست چیزی حفظ کنی — راهنما گرفتن از خود سیستم: --which، man، help و alias.

فصل ۵ — راهنما گرفتن از خود سیستم: هرگز چیزی حفظ نکن

🎯 **هدف:** یاد بگیری هر دستوری — حتی اونی که تا حالا اسمش رو نشنیدی — رو بدون گوگلِ اجباری خودت کشف کنی: --help، man، which، alias و فایل پیکربندی شل. این فصل = «ویژه‌ی فراموش‌کارها» (یعنی همه‌ی ما).

۵.۱ — سه سطح راهنما

```
$ mkdir --help          # ۱) راهنمای سریع: فلگ‌ها + یک خط توضیح هرکدوم
$ man mkdir             # ۲) کتابچه کامل (manual) — در لینوکس/مک؛ فقط Git Bash غالباً نیست
$ which node            # ۳) این دستور کجای سیستم نصبه؟
```

`--help` (یا گاهی `-h`) تقریباً روی هر ابزار مدرنی کار می‌کنه — git، npm، node، psql. عادت کن: **قبل از سرچ کردن، اول `ابزار --help` بزن.** نصف سوال‌هات همون‌جا جواب داره.

نمونه خروجی (بریده‌ای از `ls --help`):

```
Usage: ls [OPTION]... [FILE]...
  -a, --all          do not ignore entries starting with .
  -l                 use a long listing format
  -h, --human-readable  with -l, print sizes in human readable format
```

الگوی خوندنش: `Usage` می‌گه «چطور صدا زده می‌شه» (چیزی که توی `[ ]` است اختیاریه)، بعد فلگ‌ها با نسخه کوتاه/بلندشون.

۵.۲ — `which` و `type` : این دستور از کجا میاد؟

وقتی `node` رو اجرا می‌کنی، شل در یک لیست مسیرها (PATH — فصل ۹) دنبال فایل اجرایی‌اش می‌گرده. ببین کجا پیداش کرده:

```
$ which node
/c/Program Files/nodejs/node

$ which git
/c/Program Files/Git/cmd/git

$ type cd           # برای دستورهای داخلی شل (which نمی‌بیندشان)
cd is a shell builtin
```

کاربرد عملی: «چرا X کار نمی‌کنه؟» — اول `which X` . جواب خالی یعنی اصلاً نصب نیست یا تو PATH نیست (فصل ۹ دقیقاً همینو حل می‌کنه). جواب مسیر عجیب یعنی نسخه اشتباهی اجرا می‌شه!

۵.۳ — alias: اسم مستعار برای دستورهای بلند

```
$ alias cls='clear'                    # از این به بعد cls = clear
$ alias ll='ls -lah'                   # دنیا محبوب‌ترین alias
$ cls                                  # ✅ کار می‌کند
```

⚠️ مشکل: alias ای که توی ترمینال تعریف کردی، با بستن ترمینال می‌پره. راه دائمی: نوشتنش در **فایل پیکربندی شل**:

```
$ nano ~/.bashrc        # فایل پیکربندی Bash (nano = ادیتور متنی ساده ترمینال) هر بارِ اجرای
```

این خطوط رو اضافه کن، با Ctrl+O ذخیره و Ctrl+X خارج شو:

```
alias ll='ls -lah'
alias cls='clear'
alias gs='git status -sb'          # وضعیت خلاصه و تمیز گیت
alias serve='npx serve .'          # سرور استاتیک فوری
```

برای اعمال بدون ری‌استارت ترمینال:

```
$ source ~/.bashrc
```

> 💡 `~/.bashrc` = **فایلی که هر بار شل بالا میاد اول اون رو اجرا می‌کنه — جای alias ها، رنگ‌ها و تنظیمات شخصیه. مثل** `index.js` **که اول برنامه اجرا می‌شه، برای شل.**

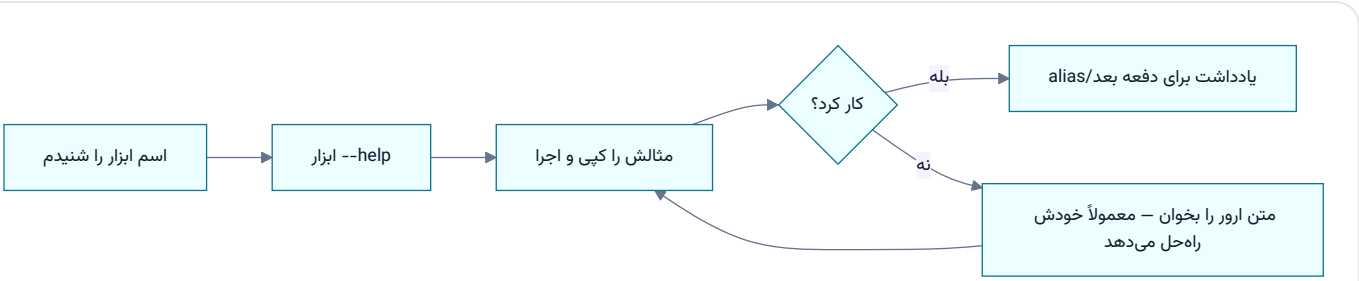
۵.۴ — راهنمای زنده: پیشنهاد خودکار

دو کمکی که Git Bash/ترمینال‌های مدرن خودکار می‌دن:

- **Tab روی فلگ‌ها:** `git comm<Tab>` کامل می‌کنه؛ حتی `git commit --<Tab><Tab>` لیست فلگ‌ها رو نشون می‌ده (اگه تکمیل گیت نصب باشه)
- **پیام «did you mean»:** `git commit` با غلط املایی → گیت خودش پیشنهاد می‌ده. خجالت نداره، همه دیر یا زود می‌بینیمش 😄

۵.۵ — استراتژی «حافظه بیرونی»

چرخه‌ای که حرفه‌ای‌ها با هر ابزار جدید انجام می‌دن:



حتی خود گیت این فلسفه رو داره:

```
$ git help commit      # کتابچه کامل زیردستور commit
$ git commit --help    # همان، در مرورگر/ادیتور
```

✅ جمع‌بندی فصل

- `ابزار --help` = اولین قدم همیشه؛ `man` کتابچه کامل (لینوکس/مک)
- `which` می‌گه فایل اجرایی از کجا اجرا می‌شه — ابزار اولیه دیباگِ «کار نمی‌کنه»
- `alias` برای خلاصه‌سازی؛ دائمی‌اش فقط در `~/.bashrc` + `source` برای اعمال فوری
- `nano` ساده‌ترین ادیتور ترمینال: Ctrl+O ذخیره، Ctrl+X خروج
- حرفه‌ای = حفظ نکردن؛ چرخه‌ی help → اجرا → alias

📝 تمرین فصل ۵

1. `node --help` و `npm --help` رو بزن — فلگ `-v` (نمایش نسخه) رو توشون پیدا کن.
2. `which` رو بزن برای: `psql` ، `npm` ، `git` ، `node` (یا هر ابزار دیتابیس دیگری که نصب داری). کدوم مسیرها عجیب‌ان؟
3. سه alias به `~/.bashrc` اضافه کن: `gs` ، `ll` (status خلاصه گیت)، `serve` . با `source` فعالشون کن و `ll` رو تست کن.
4. با nano یک فایل `about.txt` بساز، چند خط بنویس، ذخیره کن و با `cat` بخونش.

> **▼ نکته تمرین ۲**
>
> اگر `which psql` خالی برگشت ولی PostgreSQL رو نصب کردی، یعنی مسیر `C:\Program Files\PostgreSQL\18\bin` به PATH ویندوز اضافه نشده — دقیقاً همون موضوع فصل ۹! یادداشتش کن.

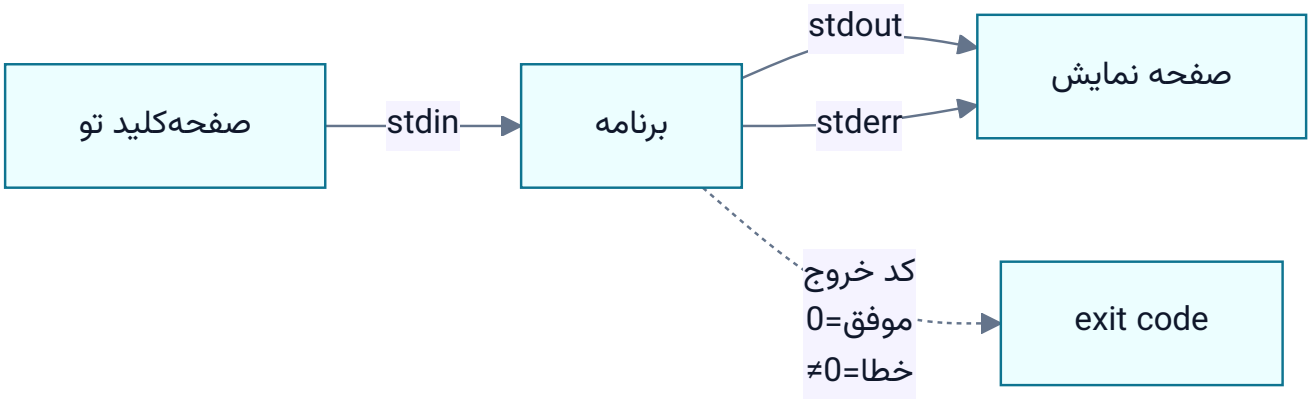
➡️ **فصل بعد:** ابرقدرت واقعی شل — اتصال دستورها به هم با pipes و ذخیره خروجی‌ها با redirection.

فصل ۶ — Pipes و Redirection: ابرقدرت شل ⭐

🎯 **هدف:** مهم‌ترین مفهوم فلسفی شل: هر برنامه یک فیلتره و خروجی هر برنامه می‌شه ورودی برنامه بعدی. با `|` و `>` و `>>` و `<` یاد می‌گیری دستورها رو مثل لگو به هم وصل کنی — همون چیزی که در فصل ۱ قولش رو داده بودیم.

۶.۱ — سه جریان: stdin، stdout، stderr

هر برنامه‌ی ترمینالی سه «لوله» داره:



جریان	شماره	چیه
stdin	0	ورودی — معمولاً صفحه‌کلید
stdout	1	خروجی عادی
stderr	2	پیام‌های خطا

خبر خوب: چون همه‌چیز *متن* است، این لوله‌ها رو می‌تونی بچرخونی و به جای صفحه نمایش، به یک برنامه دیگه یا یک فایل وصل کنی. همینه که `console.log` با نوشتن در فایل فرق داره — اینجا **جریان** قابل اتصاله.

۶.۲ — Redirection: خروجی رو بفرست جای دیگه

```
$ echo "خط اول" > notes.txt        # بنویس در فایل (بازنویسی!)
$ echo "خط دوم">>  notes.txt       # اضافه کن به انتهای فایل (append)
$ cat notes.txt                    # بخون تا مطمئن شی
خط اول
خط دوم
```

⚠️ تفاوت خطرناک `>` و `>>`: تک‌شِورون (`>`) فایل موجود رو **صفر می‌کنه**. (و راه حافظه‌ای: هرچی شِورون بیشتر، محتاط‌تر.)

ورودی هم می‌شه از فایل گرفت:

```
$ wc -l < notes.txt        # چند خط دارد؟ (ورودی از فایل)
2
```

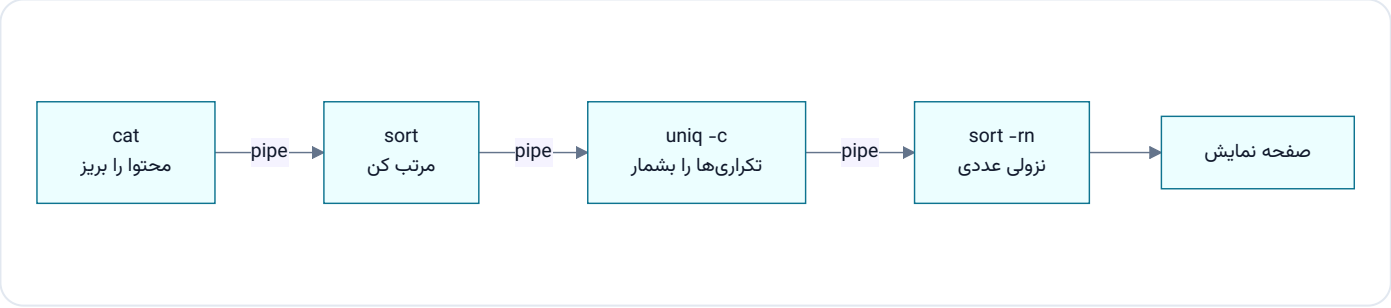
و یک ترفند که مهندس‌ها عاشقشن: خالی کردن فایل به سرعت:

```
$ > big.log        # فایل را خالی کن (به سایز صفر)
```

۶.۳ — Pipe `|`: خروجی → ورودی برنامه بعدی

قوی‌ترین کاراکتر شل:

```
$ cat orders.txt | sort | uniq -c | sort -rn
```



هر مرحله یک فیلتر ساده؛ زنجیره = یک ابزار تحلیل داده. با داده نمونه امتحانش کن:

```
$ printf "ali\nsara\nali\nreza\nsara\nali\n" | sort | uniq -c | sort -rn
  3 ali
  2 sara
  1 reza
```

چهار دستور، خروجی «رتبه‌بندی خریداران» — کوچک‌ترین داشبورد دنیا 😄

۶.۴ — ابزارهای فیلتر پایه (همتای زنجیره‌ها)

ابزار	کار	مثال
`wc`	شمارش (l=خط، w=کلمه، c=بایت)	`wc -l file` چند خط؟
`sort`	مرتب‌سازی (r=نزولی، n=عددی)	`sort names.txt`
`uniq`	فشرده‌سازی خطوط تکراری *مجاور* (همیشه با sort)	`sort f | uniq -c`
`head` / `tail`	برش اول/آخر	`... | head -5` فقط ۵ تای اول
`tee`	نمایش + ذخیره همزمان	`npm run build | tee build.log`

۶.۵ — stderr رو هم کنترل کن (سطح ۲)

خطاها به فایل نمی‌رن چون به stdout وصل نیستن! برای کنترل هر سه جریان:

```
$ npm run build > out.log            # فقط stdout ذخیره می‌شود
$ npm run build > out.log 2> err.log # خطاها جدا
$ npm run build > all.log 2>&1       # هر دو، با هم در یک فایل ⭐
$ npm run build > /dev/null 2>&1     # همه را دور بریز (dev/null = سیاه‌چاله)
```

> 💡 چرا مهمه؟ چون «لاگ بیلدم چرا فایلم خالیه؟» یکی از رایج‌ترین گیج‌شدگی‌های دولوپرهاست — جواب: خطاها در stderr بودن!

۶.۶ — سناریوهای واقعی دولوپر (اینها رو حفظ کن، هر هفته لازم می‌شن)

```
# ۱) لاگ بیلد بلنده — فقط خطاها:
$ npm run build 2>&1 | grep -i "error" | head -20

# ۲) چند فایل در این پروژه import شده از ماژول x؟
$ grep -r "from 'axios'" src/ | wc -l

# ۳) خروجی تست رو هم ببین هم ذخیره کن:
$ npm test 2>&1 | tee test-result.log

# ۴) بزرگ‌ترین پوشه‌ها/فایل‌های پروژه (دیسک چرا پر شد؟):
$ du -sh * | sort -rh | head -10

# ۵) نسخه node (مثلاً برای مستندات) بگیر و در فایل بنویس:
$ node -v > engines.txt
```

(دو تا از اینها از فصل ۷ هستن — grep و du — فعلاً فقط الگوی زنجیره رو نگاه کن.)

✅ جمع‌بندی فصل

- هر برنامه سه جریان دارد: stdin(0)، stdout(1)، stderr(2)
- `>` بازنویسی (خطرناک!)، `>>` افزودن، `<` ورودی از فایل، `> f 2>&1` = هر دو جریان
- `|` خروجی را ورودیِ بعدی می‌کند — زنجیره فیلترها = ابزار تحلیل
- فیلترهای پایه: wc، sort، uniq، head/tail، tee
- «لاگ خالی؟» = stderr جدا بود

📝 تمرین فصل ۶ (در cli-playground)

1. فایل داده بساز: `printf "قلم\nدفتر\nقلم\nموبایل\nقلم\nدفتر\n" > sales.txt`
2. با یک زنجیره pipe: تعداد فروش هر کالا رو به ترتیب نزولی نشون بده. (انتظار: قلم ۳، دفتر ۲، موبایل ۱)
3. خروجی تمرین ۲ رو در `report.txt` ذخیره کن (پایین‌ترش append هم کن: `>>` با خط جدید).
4. فایل `sales.txt` رو خالی کن با یک دستور، بعد `wc -l sales.txt` بزن (باید ۰ باشه) و بعدش از تمرین ۱ دوباره پرش کن.
5. چالش: چقدر کلمه در فایل README.md پوشه‌ات هست؟ (wc با فلگ درست + ورودی از فایل)

▼ جواب‌ها

```
# ۲
sort sales.txt | uniq -c | sort -rn
# ۳
sort sales.txt | uniq -c | sort -rn > report.txt
echo "--- گزارش تولید شد ---"  >>  report.txt
# ۵
wc -w < README.md      # یا: wc -w README.md
```

➡️ **فصل بعد:** قدرتمندترین فیلترها را جدی یاد می‌گیریم — grep برای متن و find برای فایل‌ها.

فصل ۷ — جستجو: grep و find (غول‌چراغ دولوپرها) 🔍

🎯 **هدف:** دو ابزاری که بیشترین «وقت‌صرفه‌جویی» رو تو کار روزمره دارن: `grep` برای جستجو در *محتوای* فایل‌ها و `find` برای جستجو در *درخت* فایل‌ها. آخر فصل، کلیدهای Ctrl+F پروژه‌های چند-هزار-فایلی رو داری.

۷.۱ — grep: جستجو در متن

grep = خطوطی که الگو رو دارن، چاپ کن:

```
$ grep "todo" app.js                # در یک فایل خطوط حاوی todo
$ grep -i "todo" app.js             # بدون حساسیت به بزرگ/کوچک ⭐
$ grep -n "todo" app.js             # شماره خط هم نشان بده
$ grep -r "console.log" src/        # ⭐ recursive: کل پوشه را بگرد
$ grep -c "ERROR" server.log        # فقط بشمار: چند خط؟
$ grep -v "comment" file.txt        # برعکس: خطوطی که ندارند
$ grep -A 2 -B 2 "error" log.txt    # ۲ خط قبل و بعد هم نشان بده (context)
```

خروجی `-rn` روی پروژه واقعی (پرکاربردترین شکل):

```
src/api/user.ts:14:  console.log("fetching user");
src/hooks/useAuth.ts:31:  console.log("auth state", state);
```

`فایل:خط:محتوا` — در VS Code همون خط رو Ctrl+Click کنی می‌ری سراغش. (تقریباً همون Ctrl+Shift+F خود VS Code، ولی سریع‌تر و قابل‌زنجیره با pipes.)

۷.۲ — grep در زنجیره: جایی که می‌درخشه

```
# لاگ بیلد بلند است — فقط خطاها را ببین:
$ npm run build 2>&1 | grep -i error

# کدام کامیت‌های دیروز فلان کلمه را دارند:
$ git log --oneline | grep -i "login"

# در لاگ امروز چند خط ERROR:
$ grep -c "ERROR" app.log

# همه فایل‌هایی که axios (فقط اسم فایل) مصرف می‌کنند:
$ grep -rl "axios" src/
# (-l = فقط اسم فایل‌ها، بدون خطها)
```

💡 **الگوی ذهنی:** «چیزی که دنبالشه در محتواست → grep؛ چیزی که دنبالشه یک فایله → find.»

۷.۳ — find: شکار فایل‌ها در درخت

```
$ find . -name "*.test.js"              # همه تست‌ها از همینجا به پایین
$ find . -name "*.md" -not -path "*/node_modules/*"   # بدون node_modules ⭐
$ find . -type d -name "components"     # فقط پوشه (type d)، نه فایل
$ find . -size +10M                     # فایل‌های بزرگ‌تر از ۱۰ مگ
$ find . -name "*.log" -delete          # ⚠️ حذف همه لاگ‌ها (اول با -print تست کن!)
```

- الگوی اسم باید توی کوتیشن باشه (`"*.log"`) — چون شل می‌خواد خودش بازش کنه!
- قانون ایمنی همیشگی: قبل از `-delete` ، همون find رو بدون `-delete` اجرا کن تا لیست رو ببینی.

۷.۴ — ترکیب find با ابزارهای دیگر (xargs بدون معرفی رسمی)

find خروجی‌اش اسم فایل است ← می‌شه به دستور دیگه داد:

```
# چند خط در کل فایل‌های md حاوی TODO؟
$ find . -name "*.md" -exec grep -c "TODO" {} \;

# یا خواناتر با pipe به xargs:
$ find . -name "*.js" | xargs grep -l "console.log"
```

فعلاً فقط ببین الگو: find لیست می‌سازه ← grep روش عملیات می‌کنه. (xargs = «خروجی قبلی رو به عنوان آرگومان بده».)

۷.۵ — دو ابزار مکمل که بهتره بدونی (شناختی)

ابزار	چیست	چرا بدانیش
`du -sh *`	حجم هر آیتم این پوشه	«دیسک چرا پره؟» — با `sort -rh` عالی می‌شه
`df -h`	فضای کل دیسک‌ها	قبل از دانلود بزرگ، چک کن

```
$ du -sh * | sort -rh | head -5
# 5 پوشه/فایل پرمصرف دیسک — پیدا شدن در ۲ ثانیه
```

۷.۶ — سناریوی کامل: «این تابع رو کجا تعریف و کجا صدا زدیم؟»

واقعی‌ترین موقعیت دولوپری: بازسازی/refactor یک تابع:

```
$ grep -rn "calcTotal" src/
src/utils/invoice.ts:8:   export function calcTotal(items) {
src/components/Cart.tsx:42:   const t = calcTotal(items);
src/components/Invoice.tsx:19:   return calcTotal(items);
```

در سه ثانیه فهمیدی: تعریف در invoice.ts خط ۸، دو مصرف‌کننده — و اینکه تغییر امضاش چه فایل‌هایی رو می‌شکنه.

این همون «قدرت»یه که IDE ها با ده‌ها منو می‌دن؛ CLI با یک خط.

✅ جمع‌بندی فصل

- **grep** = جستجو در محتوا: `-i` بی‌حساسیت، `-n` شماره خط، `-r` بازگشتی، `-c` شمارش، `-v` معکوس، `-l` فقط اسم فایل
- **find** = جستجو در درخت: `-name` (با کوتیشن!)، `-size` ، `-type d` ؛ قبل از `-delete` بدونش اجرا کن
- انتخاب: محتوا → grep؛ فایل → find؛ ترکیب با pipe/xargs = قدرت واقعی
- `du` و `df` برای دیباگ دیسک

📝 تمرین فصل ۷ (در cli-playground)

1. داده بساز: سه فایل js بساز (`touch a.js b.js c.js`) و با `echo "console.log(1)" > a.js` (و b و c با متن‌های مختلف — یکی شامل `error` باش) پرشون کن.
2. با grep -rn پیدا کن کدوم فایل `error` داره — با شماره خط.
3. همه فایل‌های `.js` رو با find پیدا کن، بعد همین‌و برای `.md` .
4. یک فایل بزرگ‌فایک بساز (۲۰ — `head -c 20M /dev/zero > big.bin` مگ بایت صفر!) و با find -size پیداش کن، بعد حذفش کن (اول با find بدون delete!).
5. چالش واقعی ⭐ : در یکی از پروژه‌های خود، همه جاهای استفاده از یک dependency (مثلاً axios یا clsx) رو با grep -rn پیدا کن و ببین چند فایل بهش وابسته‌ان.

▼ جواب تمرین ۱-۴

```
# ۲
grep -in "error" *.js        # (کوتیشن لازم نیست — کلمه ساده)
# ۳
find . -name "*.js"
find . -name "*.md"
# ۴
head -c 20M /dev/zero > big.bin
find . -size +10M            # لیست: ./big.bin
find . -size +10M -delete
```

➡️ **فصل بعد:** دنیای دسترسی‌ها (chmod/sudo) و اولین اسکریپت اجرایی خودت.

فصل ۸ — دسترسی‌ها، sudo و اولین اسکریپت اجرایی خودت

🎯 **هدف:** بالاخره اون `drwxr-xr-x` عجیب رو می‌فهمی (که همه‌جای خروجی `ls -l` بود)، با مفهوم sudo و ارورهای Permission denied دوست می‌شی، و اولین فایل اجرایی خودت رو می‌سازی.

۸.۱ — رمزگشایی `drwxr-xr-x`

هر خط از `ls -l` با ۱۰ کاراکتر شروع می‌شه:

```
- r w x   r - x   r - -
│ │ │ │   │ │ │   │ │ │
│ │ │ │   │ │ │   │ │ └── دیگران: خواندن و اجرا، نوشتن نه
│ │ │ │   │ │ └──────── گروه: خواندن و اجرا
│ │ │ │   └──────────── صاحب فایل: خواندن (r) نوشتن (w) اجرا (x)
```

حرف	یعنی	برای فایل	برای پوشه
`r`	read	خواندن محتوا	دیدن لیست فایل‌ها
`w`	write	تغییر محتوا	ساختن/حذف فایل داخلش
`x`	execute	اجرا به عنوان برنامه	ورود به پوشه (cd)

پس `drwxr-xr-x` یعنی: پوشه‌ای که صاحبش همه‌کار می‌تونه، بقیه فقط ببینن و وارد شن.

💡 سه‌تایی‌ها به سه گروه: **صاحب / گروه / دیگران.** همین مدل ساده، امنیت ۶۰ ساله‌ی یونیکسه — و در ویندوز هم زیر hood چیز مشابهی داره (NTFS permissions) که Git Bash برات نمایش می‌ده.

۸.۲ — chmod: تغییر دسترسی‌ها

ساده‌ترین فرم، فرم حرفی (+ اضافه، - برداشتن):

```
$ chmod +x script.sh      # اجرا-پذیرش کن برای همه (پرکاربردترین!) ⭐
$ chmod u+x script.sh     # فقط برای صاحب (u=user)
$ chmod -w secret.txt     # فقط‌خواندنی کن
```

(فرم عددی `chmod 755` و... رو بدون که وجود داره — سرورها زیادش رو می‌بینی؛ `7=rwx, 5=r-x` ولی برای کار روزمره فرم حرفی کافیه.)

۸.۳ — sudo و Permission denied

وقتی می‌بینی: `Permission denied` — یعنی کاربر فعلیت اجازه‌ی اون کار رو نداره. دو علاج:

1. **درستش کن:** دسترسی/مالکیت رو تغییر بده (`chmod`)
2. **با هویت ادمین اجرا کن:** superuser do — `sudo <دستور>`

```
$ sudo nano /etc/hosts        # فایل سیستمی را با دسترسی ادمین ویرایش کن
```

> ⚠️ دو نکته: (۱) sudo تو Git Bash ویندوز تقریباً بی‌معنیه — ویندوز خودش با پنجره‌ی «Run as UAC» («administrator همون کار رو می‌کنه؛ sudo بیشتر برای وقتیه که به سرور لینوکسی SSH می‌زنی (فصل ۱۱). (۲) قانون حرفه‌ای: **sudo رو فقط وقتی می‌زنی که واقعاً لازمه** — اجرای کورکورانه‌ی `sudo npm install` و اینا اثرات جانبی سیستمی داره.

۸.۴ — اولین اسکریپت اجرایی 🎉

حالا همه‌ی اجزا رو داری: فایل می‌سازی (فصل ۴)، متن می‌نویسی، اجرا-پذیرش می‌کنی (chmod +x). بسازیم:

```
$ mkdir -p ~/bin && cd ~/bin
$ nano hello.sh
```

محتوای فایل:

```
#!/usr/bin/env bash
# hello.sh — اولین اسکریپت من

echo "سلام $USER عزیز!"
echo "الان ساعت: $(date +%H:%M)"
echo "تو در مسیر: $PWD هستی"
```

ذخیره (Ctrl+O، Ctrl+X) و اجرا:

```
$ chmod +x hello.sh
$ ./hello.sh
سلام developer عزیز!
الان ساعت: 15:42
تو در مسیر: /c/Users/developer/bin هستی
```

چرا خط اول؟ (Shebang)

`#!/usr/bin/env bash` به سیستم می‌گه «این فایل رو با Bash اجرا کن». بدونش، ویندوز/لینوکس نمی‌دونه با چی تفسیرش کنه. (تو JS دنیا معادلش `#!/usr/bin/env node` بالای CLI-tool هاست — همونیه که npm bin ها دارن!)

چرا `./` اولش؟

چون `hello.sh` توی PATH سیستم نیست (فصل ۹!) — پس آدرس نسبی می‌دیم: «همین‌جا، این فایل».

۸.۵ — بذارش تو PATH: اسکریپت به عنوان «دستور»

اگه پوشه‌ای مثل `~/bin` رو به PATH اضافه کنی (فصل ۹ کاملش می‌کنه)، دیگه لازم نیست `./` بزنی — از هر جا مثل یک دستور عادی صدا می‌شه:

```
$ hello.sh        # بعد از افزودن ~/bin به PATH
```

الگوی کل CLI دنیا همینه: هر «دستور»ی که اجرا می‌کنی، فقط یک فایل اجراییه که تو PATH پیداش شده. `git` ، `npm` ، `node` — همه همینن!

✅ جمع‌بندی فصل

- `rwx` = خواندن/نوشتن/اجرا در سه گروه (صاحب/گروه/دیگران)؛ `x` روی پوشه = حق cd
- `chmod +x` = پرکاربردترین؛ Permission denied = یا دسترسی یا sudo (فقط لازم)
- اولین اسکریپت: shebang + chmod +x + اجرا با `./`
- هر «دستور» سیستم = فایل اجرایی در PATH — راز رازها

📝 تمرین فصل ۸

1. خروجی `ls -l` زمین تمرین رو بخون: یک فایل و یک پوشه‌ش رو تفسیر کن (کجاها w دارند؟).
2. اسکریپت `hello.sh` رو بساز و اجرا کن — بعدش دستکاریش کن که اسم پروژه‌ی محبوبت رو هم چاپ کنه.
3. دسترسی اجرا رو بردار (`chmod -x hello.sh`) و دوباره اجرا کن — ارور رو بخون، بعد برگردون.
4. یک اسکریپت `backup.sh` بساز که پوشه‌ای رو با cp -r بکاپ بگیره (چیزی که فصل ۴ دستی زدی — حالا خودکارش کن).

▼ جواب تمرین ۴

```
#!/usr/bin/env bash
# backup.sh — بکاپ از پوشه‌ای که اسمش را بدهی
cp -r "$1" "$1-backup"
echo "بکاپ گرفته شد: $1-backup"
```

(`$1` = اولین آرگومان — فصل ۱۲ کاملش می‌کند. تست: `./backup.sh shop`)

➡️ **فصل بعد:** متغیرهای محیطی و PATH — بزرگ‌ترین راز «command not found».

فصل ۹ — متغیرهای محیطی و PATH: راز «command not found»

🎯 **هدف:** پرتکرارترین سوال هر دولوپر تازه‌کار: «چرا این دستور رو نمی‌شناسه؟!». جواب در دو مفهومه: متغیرهای محیطی (که تو هر پروژه Next.js می‌بینی‌شون) و PATH (لیست جستجوی دستورها). آخر فصل این ارور برای همیشه راز نمی‌مونه.

۹.۱ — متغیر محیطی چیه؟

متغیرهای محیطی = تنظیماتی که از شل به هر برنامه‌ی اجرا شده «به ارث» می‌رسن — اسم کاربر، مسیرها، کلیدها. مثل `props` که به کامپوننت‌ها پاس می‌شه، ولی برای کل سیستم.

```
$ echo $USER            # خواندن یک متغیر: $ قبل از اسم
developer
$ echo $HOME
/c/Users/developer

$ env                   # لیست همه متغیرهای محیطی فعلی (زیاده، غلت بزن!)
```

(دقت کن: توی echo اسم با `$` شروع می‌شه ولی توی لیست env بدون `$` است — مثل تفاوت «مقدار متغیر» و «اسمش».)

۹.۲ — ساخت متغیر: دو نوع، یک تفاوت مهم

```
$ MY_VAR="سلام"         # متغیر معمولی شل — فقط در همین نشست
$ echo $MY_VAR
سلام

$ export APP_ENV="dev"  # متغیر محیطی — به برنامه‌های اجرا شده از این به بعد هم می‌رسد
```

قواعد مهم: - **بدون فاصله دور** `=` — `MY_VAR = "x"` غلطه (شل فکر می‌کنه داری برنامه‌ی MY_VAR رو اجرا کنی!) - کوتیشن‌ها رو بذار (مخصوصاً با فاصله): `APP_NAME="My Shop"` - هر دو نوع با بستن ترمینال می‌پرن — دائمی‌شون پایین‌تر

۹.۳ — دائمی کردن: ~/.bashrc

```
$ nano ~/.bashrc
```

اضافه کن:

```
export PROJECTS_DIR="/c/Users/developer/projects"    # میان‌بر پوشه پروژه‌ها
export EDITOR="code --wait"
```

بعد `source ~/.bashrc` و حالا:

```
$ cd $PROJECTS_DIR       # مستقیم به پوشه پروژه‌ها — میان‌بر سریع!
```

۹.۴ — PATH: لیست جستجوی دستورها ⭐

وقتی تایپ می‌کنی `node`، شل نمی‌دونه `node` چیه — فقط می‌ره توی مسیرهای ذخیره‌شده در متغیر **PATH** یکی‌یکی دنبال فایل اجرایی `node` می‌گرده. اولین جایی که پیداش کنه اجرا می‌کنه:

```
$ echo $PATH
/c/Program Files/nodejs:/c/Program Files/Git/cmd:/c/Users/developer/bin:...
```

مقادیر با `:` (لینوکس/مک) یا `;` (ویندوز) جدا می‌شن. حالا این جمله رو قاب کن:

> **«command not found» یعنی: در هیچ‌کدوم از پوشه‌های PATH فایل اجرایی با این اسم نبود.**

دو علاج همیشه: 1. **نصبش ناقصه** → نصبش کن 2. **نصبه ولی مسیرش توی PATH نیست** → مسیرش رو به PATH اضافه کن:

```
$ export PATH="$PATH:/c/Program Files/PostgreSQL/18/bin"
$ psql --version          # ✅ حالا از هر جا کار می‌کند
```

(دقت کن الگو: `PATH="$PATH:مسیرجدید"` — یعنی «PATH قبلی + یک پوشه جدید». اگه `$PATH:` رو جا بندازی، بقیه مسیرها از بین می‌رن و کل ترمینال دیوانه می‌شه! دائمی‌اش در `~/.bashrc`.)

و راه ویندوز-خالص (برای همه برنامه‌ها حتی خارج از Git Bash): Start → «environment variables» → Edit the system environment variables → Environment Variables → ویرایش New → `Path` → مسیر. بعد ترمینال رو ببند و دوباره باز کن (ترمینال‌های باز PATH قدیمی رو دارن).

۹.۵ — فایل‌های .env: همون مفهوم، در مقیاس پروژه

چیزی که تو هر پروژه Next.js می‌بینی — `.env` — همون متغیر محیطیه، فقط *محلی برای یک پروژه*:

```
# .env (پروژه my-shop)
DATABASE_URL="postgresql://postgres:123@localhost:5432/shop"
NEXT_PUBLIC_API_URL="https://api.myshop.dev"
```

Next.js موقع اجرا اینها رو می‌خونه و توی `process.env.DATABASE_URL` می‌ذاره. چرا اینطور؟ چون: - هر پروژه تنظیمات خودش رو داره (لوکال ≠ پروداکشن) - **رازها توی کد نمی‌ان** — فایل `.env` باید حتماً در `.gitignore` قرار گیرد تا امنیت حفظ شود

> 🔑 یک مفهوم، سه سطح: متغیر شل (نشست فعلی) → export در bashrc (همه نشست‌های تو) → فایل .env (محلی برای یک پروژه). دولوپر واقعی بین این سه سطح جابجا می‌شه.

۹.۶ — چند متغیر معروفی که هرجا می‌بینی

متغیر	چیه
`PATH`	لیست مسیرهای جستجوی دستورها
`HOME`	پوشه خانه کاربر
`USER`	اسم کاربر فعلی
`EDITOR`	ادیتور پیش‌فرض (git برای پیام کامیت ازش می‌پرسه)
`NODE_ENV` / `DATABASE_URL`	قراردادهای دنیای Node

✅ جمع‌بندی فصل

- متغیر محیطی = تنظیماتی که به برنامه‌ها ارث می‌رسه؛ خواندن با `echo $NAME` ، لیست با `env`
- ساخت: `NAME="value"` (بدون فاصله!)؛ دائمی: `export` در `~/.bashrc` + `source`
- **PATH = لیست جستجو.** «command not found» = اسمش تو PATH نیست. علاج: `export PATH="$PATH:/مسیر/جدید"`
- env. پروژه = همون مفهوم در مقیاس محلی، برای جداسازی تنظیمات و رازها از کد

📝 تمرین فصل ۹

1. سه متغیر `SHELL` ، `HOME` ، `USER` رو با echo چاپ کن.
2. متغیر `GREETING="سلام مهندس"` بساز، echo بزن، ترمینال رو ببند و باز کن — پریده؟ چرا؟
3. دائمی: یک متغیر `WORK="مسیر-زمین-تمرین"` رو در export `~/.bashrc` کن، source بزن و با `cd $WORK` مستقیم برو زمین تمرین.
4. `echo $PATH` رو بزن و سه تا از مسیرهاش رو بگو چی‌ان (به چی مربوطن؟).

5. (کاربردی ⭐) اگه `psql` روی سیستمته ولی در ترمینال نیست، مسیر `C:\Program Files\PostgreSQL\18\bin` رو به PATH اضافه کن (همون الگوی فصل) و تستش کن.

▼ جواب تمرین ۲ و ۳

```
# ۲: باید؛ زنده‌اند نشست همان در فقط معمولی متغیرهای – پریده بله export در bashrc شوند.
# ۳:
echo 'export WORK="/c/Users/developer/cli-playground"' >> ~/.bashrc
source ~/.bashrc
cd $WORK
```

(ترفند `>>` برای اضافه‌کردن به فایل از فصل ۶ — بد نیست یکبار هم با nano دستی بنویسی!)

➡️ **فصل بعد:** پروسه‌ها — وقتی `npm run dev` رو می‌زنی دقیقاً چی اجرا می‌شه و چطوری نگهش داری/بکشی/بکشیش کنی.

فصل ۱۰ — پروسه‌ها: اجرا، توقف، پس‌زمینه و آزاد کردن پورت ۳۰۰۰

> 🎯 **هدف:** هر دستوری که اجرا می‌کنی یک «پروسه» می‌سازه. یاد می‌گیری پروسه‌ها رو ببینی، در پس‌زمینه اجرا کنی، متوقف کنی — و معروف‌ترین کاربردش: وقتی Node روی پورت ۳۰۰۰ گیر کرده و نمی‌دونی چرا!

۱۰.۱ — پروسه چیه؟

هر برنامه در حال اجرا = یک **پروسه (Process)** با یک شناسه یکتا به اسم **PID**. `npm run dev` تو یک پروسه‌ست، همون ترمینال بازت هم یک پروسه‌ست. ببینشون:

```
$ ps           # پروسه‌های همین ترمینال
    PID TTY          TIME CMD
   1234 cons0    00:00:01 bash
   5678 cons0    00:00:00 ps

$ ps -ef | head -15      # همه پروسه‌های سیستم (در Git Bash)
```

ستون PID رو خوب یادت بمونه — کلید کنترل پروسه‌هاست.

۱۰.۲ — پیش‌زمینه، پس‌زمینه و سه کلید مهم

وقتی `npm run dev` می‌زنی، ترمینال «قفل» می‌شه چون برنامه داره جلوی چشمت اجرا می‌شه (foreground). کلیدهای کنترل:

کلید/دستور	کار	سناریو
`Ctrl+C`	توقف برنامه جلویی (SIGINT)	سرور رو می‌بندی، اسکن طولانی رو لغو می‌کنی
`Ctrl+Z`	«نگه‌دار» — برنامه می‌ره خواب موقت	«صبر کن وسط کارم یه چیزی بزنم»
`bg`	خواب رو بفرست به پس‌زمینه	ادامه‌اش بده بدون دیدن خروجی
`fg`	برگردون به جلوی صفحه	دوباره خروجی‌اش رو می‌خوام
`jobs`	لیست کارهای پس‌زمینه‌ی همین ترمینال	«چی دارم تو پس‌زمینه؟»

و میان‌برِ راحت: `&` آخر دستور = مستقیم به پس‌زمینه:

```
$ sleep 60 &              # ثانیه بخواب — ولی منتظرت نمی‌مانم ۶۰
[1] 4321                  # و [1] شماره کار PID 4321
$ jobs
[1]+  Running    sleep 60 &
```

💡 **نکته ویندوزی:** چون هر `npm run dev` در ترمینال خودش جلو می‌ره، دولوپرهای ویندوز معمولاً چند *تب* ترمینال باز می‌کنن (سرور/تست/دستورات). اونجا که ترمینال گرافیکی داری، `&` کمتر لازم می‌شه — ولی مفهومش رو که بلد باشی روی سرور لینوکسی (SSH — فصل ۱۱) طلاست.

۱۰.۳ — kill: بستن پروسه با PID

```
$ kill 4321           # درخواست تمیز: پروسه 4321 بسته شو
$ kill -9 4321        # ⚠️ (وقتی جواب نمی‌دهد — SIGKILL) قتل!
```

- `kill` پیش‌فرض *درخواست* می‌فرسته (مثل window close) — برنامه می‌تونه تمیز جمع بشه
- `-9` یعنی «سسیستم بهش بکشه» — آخرین راه، وقتی برنامه دیگه جواب نمی‌ده

پیدا کردن PID چیزی که می‌خوای:

```
$ ps -ef | grep node          # همه پروسه‌های node
$ tasklist | grep -i chrome   # راه ویندوزی (tasklist = مدیر وظایف متنی)
```

۱۰.۴ — ⭐ سناریوی ستاره‌ی فصل: پورت ۳۰۰0 در اشغال است!

این ارور رو حتماً تا حالا دیدی یا خیلی زود می‌بینی:

```
Error: listen EADDRINUSE: address already in use :::3000
```

یعنی یه پروسه‌ی قدیمی (معمولاً dev server قبلی که درست بسته نشد) هنوز روی پورت ۳۰۰۰ نشسته. راه‌حل قدم‌به‌قدم:

```
# ویندوز (در cmd یا PowerShell یا Git Bash):
$ netstat -ano | grep ":3000"
  TCP    0.0.0.0:3000     0.0.0.0:0      LISTENING       12345          ← PID پیدا شد!

$ taskkill //PID 12345 //F
SUCCESS: The process with PID 12345 has been terminated.
```

⚠️ **دقت:** در **Git Bash** اسلش‌های فلگ دوتایی می‌شن (`//F` `//PID`) چون تک‌اسلش مسیر ویندوزی تفسیر می‌شه! در PowerShell/CMD عادی: `taskkill /PID 12345 /F` .

```
# لینوکس/مک (و سرورها):
$ lsof -i :3000        # یا: ss -tlnp | grep 3000
$ kill -9 <PID>
```

این چرخه رو یکبار خودت انجام بدی، دیگه عمراً از این ارور نترسی. 😎

۱۰.۵ — نظارت زنده: چی داره منابع می‌خوره؟

ابزار	کجا	کار
`top`	لینوکس/مک Git Bash	مانیتور زنده متنی (q = خروج)
`htop`	لینوکس/مک (نصبی)	نسخه قشنگ‌تر و قابل‌کار top
Task Manager	ویندوز (Ctrl+Shift+Esc)	همون ولی گرافیکی

```
$ top
# خروج q، بر اساس رم M، مرتب بر اساس CPU  P :کلیدهای مفید داخلش
```

سناریوی واقعی: «فنم داره می‌ترکه!» → `top` → مرتب بر اساس CPU → مقصر رو `kill` کن.

۱۰.۶ — exit code: کد خروج هر دستور

هر پروسه موقع تموم شدن یک عدد می‌ذاره: `0` = موفق، غیرصفر = خطا. دیدنش:

```
$ ls
$ echo $?
0                      # ls موفق بود

$ ls /nejad
ls: cannot access...     # خطا
$ echo $?
2
```

فعلاً فقط بدونش هست؛ در اسکریپت‌نویسی (فصل ۱۲) شرط‌ها بر اساس همین عددن — و پایپ‌لاین‌های اتوماسیون و CI/CD نیز دقیقاً با همین منطق تصمیم‌گیری می‌کنند: «سبز = exit code صفر».

✅ جمع‌بندی فصل

- هر برنامه در حال اجرا = پروسه با PID؛ دیدن با `ps -ef`
- `Ctrl+C` توقف جلویی، `&` و `bg/fg/jobs` برای پس‌زمینه
- `kill` درخواست تمیز؛ `kill -9` آخرین راه
- **EADDRINUSE**: `netstat -ano | grep :3000` → `taskkill //PID n //F` (در Git Bash دوتایی!)

- `top` برای نظارت؛ exit code صفر = موفق

📝 تمرین فصل ۱۰

1. `sleep 100` بزن و با Ctrl+C وسطش لغوش کن.
2. `sleep 100 &` بزن، با `jobs` ببینش، با `kill %1` (٪ یعنی شماره job نه PID!) بکشش، دوباره jobs بزن.
3. خروجی `ps -ef | grep -i node` رو ببین — الان چند پروسه node داری؟ (اگه dev server روشنه حتماً می‌بینی‌ش!)
4. چالش اصلی ⭐: عمداً یک dev server روشن کن (`npx serve .` یا `npm run dev` در یک پروژه)، از یک ترمینال دیگه با netstat پیداش کن، با taskkill ببندش — و ببین سرورِ ترمینال اول چه پیامی داد.
5. `echo $?` رو بعد از یک دستور موفق و یک دستور شکست‌خورده مقایسه کن.

▼ جواب تمرین ۲

```
sleep 100 &          # [1] 4321
jobs                 # [1]+  Running  sleep 100 &
kill %1              # %1 = job شماره ۱ (نه PID!)
jobs                 # [1]+  Terminated  sleep 100
```

➡️ **فصل بعد:** جعبه‌ابزار روزمره‌ی دولوپر — node/npm/npx، curl، ssh و بقیه دوستایی که هر روز می‌بینی.

فصل ۱۱ — جعبه‌ابزار روزمره‌ی دولوپر: node/npm/npx، curl، ssh و دوستان

> 🎯 **هدف:** ابزارهایی که تو هر پروژه واقعی و هر شرکت می‌بینی و شاید تا حالا بدون آگاهی عمیق استفاده می‌کردی — حالا سازوکار و کاربرد دقیق هر کدوم رو یاد می‌گیری و مسلط می‌شی.

۱۱.۱ — node، npm، npx: سه‌قلوهای دنیای جاوااسکریپت

این سه ابزار پایه‌ی توسعه جاوااسکریپت و وب هستند؛ تفاوت‌های دقیق آنها:

ابزار	چیه	مثال
`node`	موتور اجرای JS بیرون از مرورگر	`node script.js`
`npm`	مدیر پکیج‌ها (نصب از npmjs.com)	`npm install axios`
`npx`	«اجرا کن (اگه نبود اول بگیر)»	`npx create-next-app@latest`

```
$ node -v                # نسخه موتور
v24.19.0

$ node                   # REPL — مثل کنسول مرورگر، تعاملی!
> 2 + 2
4
> .exit                  # خروج (یا Ctrl+C دو بار / Ctrl+D)

$ npm -v                 # نسخه npm
11.19.0
```

نکته‌ی npx که خیلی‌ها نمی‌دونن: اون `@latest` آخر اسم یعنی «آخرین نسخه رو بگیر اجرا کن» — و npx اول چک می‌کنه تو `node_modules` پروژه‌ات داریش یا نه. برای همینه که `npx create-next-app` بدون نصب دائمی کار می‌کنه.

فلگ‌های npm که باید بشناسی:

```
$ npm install            # نصب طبق package.json (خلاصه: npm i)
$ npm run dev            # اجرای اسکریپت dev از package.json
$ npm test               # میان‌بر run test
$ npm install -g pnpm    # -g = نصب سراسری (global) — روی سیستم، نه فقط پروژه
```

⚠️ `-g` رو کم و محتاط استفاده کن — هر چیزی که global نصب می‌شه، نسخه‌ی «سیستمی» می‌شه و بین پروژه‌ها ممکنه تداخل بده. `npx` اغلب جایگزین بهتریه چون دائم تازه نگهش می‌داره.

۱۱.۲ — curl: پستچی اینترنت

curl = یک HTTP request از ترمینال. وقتی API می‌سازی و می‌خوای بدون مرورگر/Postman چکش کنی:

```
$ curl https://api.github.com/zen
Keep it logically awesome.          # خروجی: متن ساده

$ curl -i https://api.github.com/zen    # با هدرها (-i = include)
$ curl -o page.html https://example.com # ذخیره در فایل (-o = output)
```

و فرم ارسال داده (که در مستندات هر API دنیا می‌بینی):

```
$ curl -X POST https://api.example.com/users \
    -H "Content-Type: application/json" \
    -H "Authorization: Bearer MY_TOKEN" \
    -d '{"name": "ali"}'
# -X متد، -H هدر، -d بدنه داده
```

چرا بهتر از مرورگر برای دیباگ API؟ چون *دقیقاً* می‌بینی چی می‌فرستی و چی میاد — بدون کش و کوکی و جادوی مرورگر.

۱۱.۳ — ssh و scp: کنترل از راه دور

پروتکل SSH کلید ورود امن به دنیای شبکه است: **ورود به هر سرور لینوکسی دنیا با ترمینال خودت.**

```
$ ssh user@203.0.113.10
# مثل اینکه کیبوردت به یک سرور وصل شده — همه دستورهای این دوره اونجا هم کار می‌کنند!

$ exit                          # برگشت
```

دو ابزار مکمل:

```
$ scp file.zip user@203.0.113.10:/home/user/     # کپی فایل به سرور (cp روی شبکه!)
$ ssh user@host "uptime"                         # یک دستور، بدون ورود کامل
```

وقتی روزی به سرور production وصل شی (با ssh)، همه‌ی این دوره یعنی: `tail -f` ، `cd` لاگ، `ps` و `grep` ، `top` در لاگ‌ها، `nano` تنظیمات. برای همینه که می‌گم CLI = زبان مشترک سرورها.

۱۱.۴ — آرشیو: tar و zip

پکیج‌های لینوکسی و بکاپ‌ها معمولاً `.tar.gz` ان:

```
$ tar -czf backup.tar.gz my-folder/     # فشرده‌سازی: c=ساخت, z=فشرده gzip, f=اسم فایل
$ tar -tzf backup.tar.gz                # محتواش را لیست کن (بدون باز کردن)
$ tar -xzf backup.tar.gz                # باز کردن: x=extract
```

قانون حفظی محبوب: **«c برای ساختن، x برای بیرون آوردن»** — فقط این دو تا یادت باشه، بقیه از --help میاد.

۱۱.۵ — wget و download (مکمل curl)

```
$ wget https://example.com/file.zip     # دانلود مستقیم فایل
$ curl -O https://example.com/file.zip  # معادل با curl (-O = ذخیره با اسم اصلی)
```

۱۱.۶ — جدول جمع‌بندی: ابزارهای پرکاربرد اکوسیستم دولوپری

ابزار	کارایی اصلی	زمینه کاربرد
`npx` / `npm` / `node`	اجرای رانتایم / مدیریت پکیج / اجرای موقت ابزارها	پروژه‌های جاوااسکریپت و فول‌استک
`gh` / `git`	کنترل نسخه کد / تعامل با مخازن از ترمینال	مدیریت سورس کد و ریپازیتوری
`psql`	کلاینت خط فرمان دیتابیس	مدیریت پایگاه داده و اجرای کوئری
`ssh` / `ssh-keygen`	ساخت کلیدهای امنیتی / اتصال امن از راه دور	اتصال به سرورها و احراز هویت
`curl`	ارسال درخواست‌های HTTP و دریافت پاسخ	تست API و ارتباطات شبکه
`npx serve` / `npm run dev`	راه‌اندازی وب‌سرور لوکال	محیط توسعه فرانت‌اند و بک‌اند

✅ جمع‌بندی فصل

- `node` موتور، `npm` فروشگاه، `npx` اجرای لحظه‌ای (`@latest` = تازه‌ترین)
- `curl` پستچی HTTP: `-i` هدرها، `-X POST -H -d` ارسال داده — دیباگ API بدون مرورگر
- `ssh` = ورود به هر سرور دنیا؛ همه‌ی این دوره روی سرور هم کار می‌کند؛ `scp` کپی روی شبکه
- `tar -czf` بساز / `-xzf` باز کن
- اکوسیستم دولوپری یک زبان مشترک داره: همینه که یاد گرفتی

📝 تمرین فصل ۱۱

1. با `node` یک REPL باز کن، چند محاسبه کن و با `.exit` بیا.
2. `npm -v` و `npx -v` و `node -v` رو مقایسه کن (چرا npm و npx معمولاً هم‌نسخه‌ان؟).

3. با curl از `https://api.github.com/zen` یک جمله‌ی حکیمانه بگیر — چند بار بزن، جمله‌ها عوض می‌شن!

4. curl با `-i` بزن به یک سایت و هدرهای پاسخ رو بخون (`content-type` چیه؟).

5. یک پوشه رو tar کن، محتواش رو با `-tzf` ببین، حذفش کن، با `-xzf` برگردون — چرخه کامل آرشیو.

<details>
<summary>▼ جواب تمرین ۲ و ۵</summary>

```
# ۲: چون npm و npx هر دو همراه node خودشان نصب می‌شوند (part of npm package).
# ۵:
tar -czf demo.tar.gz cli-playground/     # ساخت
tar -tzf demo.tar.gz | head -5           # لیست
rm -r cli-playground && tar -xzf demo.tar.gz   # بازیابی!
```

</details>

➡️ **فصل بعد:** اسکریپت‌نویسی — همه‌ی این دستورات را به «دکمه‌های شخصی خودت» تبدیل کن.

فصل ۱۲ — اسکریپت‌نویسی Bash: خودکارسازی کارهای تکراری

🎯 **هدف:** فصل ۸ اولین اسکریپتت رو ساخت؛ حالا یاد می‌گیری برنامه‌نویسی واقعی کنه: متغیر، آرگومان، شرط، حلقه. آخر فصل یک اسکریپت کاربردی کامل داری که هر روزه استفاده‌ش می‌کنی.

۱۲.۱ — متغیر و آرگومان‌ها

```
#!/usr/bin/env bash
NAME="مهندس"                    # تعریف: بدون فاصله دور =
echo "سلام $NAME"               # استفاده: $ قبل از اسم

echo "اسم اسکریپت: $0"          # متغیرهای خاص
echo "آرگومان اول: $1"          # آرگومان‌های ورودی.. $1 $2 $3
echo "تعداد آرگومان‌ها: $#"
echo "همه آرگومان‌ها: $@"
```

اجرا:

```
$ ./myscript.sh ali reza
سلام مهندس
اسم اسکریپت: ./myscript.sh
آرگومان اول: ali
تعداد آرگومان‌ها: 2
همه آرگومان‌ها: ali reza
```

و قرارداد دوست‌داشتنی: مقدار پیش‌فرض با `${VAR:-پیش‌فرض}` :

```
TARGET="${1:-project}"      # اگر 1$ داده نشده بود، "project"
echo "هدف: $TARGET"
```

⚠️ **قانون طلایی:** در echo ها و جابجایی‌ها متغیرها رو توی **کوتیشن دوتایی** بذار (`"$FILE"`) — تا اسم‌های دارای فاصله یا خالی‌ها اسکریپت رو نشکنن.

۱۲.۲ — ورودی کاربر: read

```
#!/usr/bin/env bash
read -p "اسم پروژه‌ات چیه؟ " PROJECT
echo "در حال ساخت $PROJECT..."
mkdir -p "$PROJECT"
echo "✅ ساخته شد!"
```

`-p` یعنی «قبل از خوندن، این پیام رو نشون بده». مثل `prompt()` جاوااسکریپت، ولی برای شل.

۱۲.۳ — شرط‌ها: if

```bash
#!/usr/bin/env bash
FILE="$1"

if [ -f "$FILE" ]; then
  echo "$FILE یک فایل معمولی است — ${ } :$(wc -l < "$FILE") خط"
elif [ -d "$FILE" ]; then
  echo "$FILE یک پوشه است"
else
  echo "$FILE وجود ندارد!"
  exit 1                        # کد خروج غیرصفر = شکست (بررسی‌شده در فصل ۱۰)
fi
```

عملگرهای تست رایج:

تست	معنی
`-f x` / `-d x`	فایل هست؟ / پوشه هست؟
`-z "$s"` / `-n "$s"`	خالی است؟ / غیرخالی است؟
`"$a" = "$b"`	برابرند؟
`$n -eq 0` / `-gt 5` / `-lt 3`	برابر / بزرگ‌تر / کوچک‌تر (اعدادی)

⚠️ جزئیات نحوی که همه گیرش می‌افتن: اطراف `[ ]` **فاصله اجباریه** (`[ -f "$f" ]` نه `[-f "$f"]`)، و `then` و `fi` (برعکسِ if) یادت نره.

```bash
# یک‌خطی هم می‌شود (&& اگر موفق بود بعدی، || اگر شکست خورد بعدی):
[ -d node_modules ] && echo "نصب شده" || npm install
```

۱۲.۴ — حلقه‌ها: for

```bash
#!/usr/bin/env bash
# برای هر فایل md یک خروجی txt بساز
for f in docs/*.md; do
  echo "پردازش: $f"
  cp "$f" "${f%.md}.txt"        # ${f%.md} = اسم بدون پسوند .md
done

# روی لیست ثابت:
for name in ali sara reza; do
  echo "سلام $name"
done

# شمارشی:
for i in 1 2 3 4 5; do echo "خط $i"; done
```

ترکیب with find (فصل ۷): همه فایل‌های tmp رو پیدا و حذف کن — ولی امن:

```bash
for f in $(find . -name "*.tmp"); do
  echo "حذف $f"
  rm "$f"
done
```

۱۲.۵ — توابع و کامل‌ترین اسکریپت دوره: `newproj.sh` ⭐

بذار همه‌چیز رو جمع کنی — اسکریپت ساخت پروژه‌ی جدید:

```bash
#!/usr/bin/env bash
# newproj.sh — ساخت اسکلت پروژه با یک دستور
# استفاده: ./newproj.sh <اسم> [تایپ]

NAME="${1:-}"
TYPE="${2:-basic}"

if [ -z "$NAME" ]; then
  echo "❌ استفاده: ./newproj.sh <اسم پروژه> [basic|web]"
  exit 1
fi

create_basic() {
  mkdir -p "$NAME/src" "$NAME/docs"
  echo "# $NAME" > "$NAME/README.md"
}

create_web() {
  mkdir -p "$NAME/src/components" "$NAME/public"
  cat > "$NAME/README.md" <<EOF
# $NAME
پروژه وب ساخته‌شده با newproj.sh
EOF
}

case "$TYPE" in
  web)   create_web ;;
  basic) create_basic ;;
  *)     echo "❌ تایپ ناشناخته: $TYPE"; exit 1 ;;
esac

echo "✅ پروژه $NAME ($TYPE) ساخته شد در: $(pwd)/$NAME"
```

سه چیز جدید که اینجا دیدی:

ساختار	کار
`cat > file <<EOF ... EOF`	heredoc — نوشتن چندخطی در فایل
`case ... esac`	سوئیچ کیس شل (ساختار دسته‌بندی ورودی‌ها)
توابع با `name() { }`	مثل توابع JS — با صدازدن اسمش

۱۲.۶ — اسکریپت‌ها رو «دستور» کن

الگوی حرفه‌ای (ادامه‌ی فصل ۸): پوشه `~/bin` بساز، اسکریپت‌های کاربردی‌ات رو اونجا بذار، `chmod +x` بزن و مسیرش رو به PATH اضافه کن (فصل ۹):

```
$ mkdir -p ~/bin
$ nano ~/bin/newproj.sh        # (محتوای بالا)
$ chmod +x ~/bin/newproj.sh
$ echo 'export PATH="$PATH:/c/Users/developer/bin"' >> ~/.bashrc
$ source ~/.bashrc
$ cd ~ && newproj.sh test-shop web     # 🎉 از هر جا! مثل یک دستور رسمی سیستم
```

حالا اسکریپت‌های شخصی تو، هم‌ردیف `git` و `npm` نشستن.

✅ جمع‌بندی فصل

- متغیر بدون فاصله دور `=` ؛ آرگومان‌ها `$1 $2...` ؛ پیش‌فرض `${1:-پیش‌فرض}`
- کوتیشن دوتایی دور متغیرها = ضد ترکیدن اسکریپت
- `if [ -f x ]; then ... fi` — فاصله‌های `[ ]` اجباری؛ `||` / `&&` برای یک‌خطی‌ها
- `for` روی wildcard ها و لیست‌ها؛ `read -p` برای ورودی کاربر
- heredoc برای متن چندخطی؛ `case/esac` برای دسته‌بندی؛ توابع برای تمیزی
- مسیر حرفه‌ای: اسکریپت‌های کاربردی در `~/bin` + PATH → «دستورهای شخصی»

📝 تمرین فصل ۱۲

1. اسکریپت `greet.sh` بنویس که اسم رو از آرگومان بگیره (بدون آرگومان: «همه» سلام کنه) — از `${1:-همه}` استفاده کن.
2. `check.sh <مسیر>` بنویس: اگه پوشه بود «پوشه‌ست»، فایل بود «فایله، X خط»، نبود «وجود نداره» با exit 1.
3. حلقه‌ای بنویس که همه فایل‌های `.txt` زمین تمرین رو بشماره و «نام‌فایل: تعداد-کلمه» چاپ کنه (راهنمایی: `wc -w < "$f"` داخل حلقه).
4. `newproj.sh` رو کامل بساز، به `~/bin` بده و دو پروژه باهاش بساز (basic و web) — و `ls` روی هر دو بزن.
5. چالش ⭐ : اسکریپت `cleanup.sh` بنویس که: از پوشه‌ی داده‌شده بکاپ بگیره (tar)، فایل‌های `.tmp` رو حذف کنه و آخرش گزارش بده. (ترکیب فصل‌های ۴، ۷، ۱۱!)

▼ جواب تمرین ۵

```bash
#!/usr/bin/env bash
DIR="${1:-.}"
tar -czf "$(basename "$DIR")-backup.tar.gz" "$DIR"
find "$DIR" -name "*.tmp" -type f -print -delete
echo "✅ بکاپ گرفته شد و tmp ها پاک شدند."
```

(تستش رو اول روی یک پوشه‌ی تمرینی بزن، نه پروژه‌ی واقعی!)

➡️ **فصل آخر:** چیت‌شیت کامل + گلاساری + نقشه‌ی ادامه‌ی راه.

فصل ۱۳ — چیت‌شیت، گلاساری و راه ادامه

🎯 **هدف:** مرجع مرور سریع دوره — روی دیوار، کنار کیبورد. چاپش کن.

۱۳.۱ – چیت‌شیت دستورات (به ترتیب اهمیت روزمره)

```
# ═══ ناوبری (فصل ۳) ═══
pwd                          # کجام؟
ls -lah                      # لیست کامل خوانا
cd <مسیر> / cd .. / cd / cd -   # برو / بالا / خانه / برگشت
code .                       # پوشه فعلی در VS Code

# ═══ فایل‌ها (فصل ۴) ═══
mkdir -p a/b/c               # پوشه تو در تو
touch f.txt                  # فایل خالی
cp -r a b / mv a b           # کپی / جابجایی+نام
rm f / rm -r dir/            # حذف (⚠️ اول ls با همون مسیر!)
cat f / less f / tail -f log # خواندن / پیمایش / دنبال زنده

# ═══ راهنما (فصل ۵) ═══
ابزار-- help                 # اولین کار همیشه!
which node                   # از کجا اجرا می‌شود؟
alias ll='ls -lah'           # در ~/.bashrc + source

# ═══ جریان (فصل ۶) ═══
cmd > f  /  cmd >> f         # ذخیره بازنویسی / افزودن
cmd1 | cmd2                  # خروجی ← ورودی
cmd > all.log 2>&1           # خطاها هم ذخیره
> big.log                    # خالی کردن فایل

# ═══ جستجو (فصل ۷) ═══
grep -in "کلمه" فایل/پوشه    # -r بازگشتی، -n شماره خط، -c شمارش
find . -name "*.js"          # (کوتیشن!) قبل از -delete بدون delete بزن
du -sh * | sort -rh          # پرحجم‌ترین‌ها

# ═══ دسترسی و اسکریپت (فصل ۸) ═══
chmod +x script.sh           # اجرا-پذیر کن
./script.sh                  # اجرا (~/bin + PATH = از هر جا)

# ═══ متغیرها (فصل ۹) ═══
echo $VAR  /  env            # خواندن / لیست
export VAR="x"               # ساخت محیطی (در ~/.bashrc دائمی)
export PATH="$PATH:/جستجو مسیر افزودن #     "مسیر

# ═══ پروسه‌ها (فصل ۱۰) ═══
ps -ef | grep node           # پروسه‌ها
Ctrl+C  /  Ctrl+Z + bg/fg    # توقف / پس‌زمینه
cmd &                        # اجرا در پس‌زمینه
kill PID / kill -9 PID       # بستن / قتل
netstat -ano | grep :3000    # چه کسی پورت را گرفته؟
taskkill //PID n //F         # (Git Bash!) / در cmd: /PID /F
echo $?                      # کد خروج دستور قبل (0=موفق)

# ═══ ابزارها (فصل ۱۱) ═══
node / npm run dev / npx pkg # اجرا / اسکریپت / یک‌بارمصرف
curl [-i] [-X POST -H -d]    # HTTP دستی
ssh user@host  /  scp f h:p  # ورود ریموت / کپی ریموت
tar -czf a.tar.gz dir/  /  tar -xzf a.tar.gz

# ═══ اسکریپت (فصل ۱۲) ═══
#!/usr/bin/env bash          # خط اول همیشه
"$1" / ${1:-پیش‌فرض}          # آرگومان / با پیش‌فرض
if [ -f "$x" ]; then ... fi  # اجباری [ ] فاصله‌های!
for f in *.txt; do ...; done
```

۱۳.۲ – کلیدهای مقدس

کلید	کار
`Tab` / `Tab Tab`	کامل‌سازی / لیست گزینه‌ها
`↑` `↓`	تاریخچه
`Ctrl+R`	جستجوی تاریخچه
`Ctrl+C`	لغو / فرار
`Ctrl+L`	پاک کردن صفحه
`Ctrl+A` / `Ctrl+E`	اول / آخر خط
`Ctrl+U`	پاک کردن تا اول خط

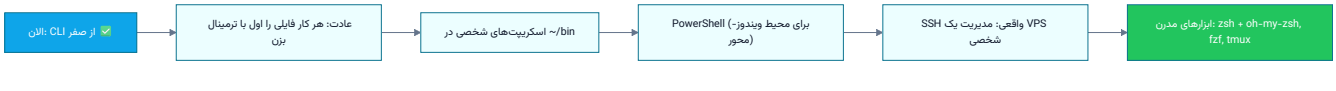
۱۳.۳ — گلاساری فارسی–انگلیسی

انگلیسی	فارسی	یک جمله
Terminal	ترمینال	پنجره‌ای که شل داخلش اجرا می‌شود
Shell (Bash)	شل	مترجم دستورات متنی
Working Directory	پوشه کاری	جایی که دستورات نسبت به آن اجرا می‌شوند
Absolute/Relative Path	مسیر مطلق/نسبی	از ریشه / از جای فعلی
Flag / Option	فلگ/گزینه	تنظیم رفتار دستور (`-r` , `--all`)
Argument	آرگومان	موضوع دستور (اسم فایل و...)
Wildcard / Glob	الگوی عام	`*.log` و `?` برای کار گروهی
Pipe	لوله `|`	اتصال خروجی به ورودی
stdin/stdout/stderr	ورودی/خروجی/خطا	سه جریان هر برنامه (0/1/2)
Redirection	بازهدانی	`>` و `>>` و `<`
Environment Variable	متغیر محیطی	تنظیماتی که به برنامه‌ها ارث می‌رسد
PATH	—	لیست مسیرهای جستجوی دستورها
Process / PID	پروسه / شناسه پروسه	برنامه در حال اجرا و شماره‌اش
Exit Code	کد خروج	۰ = موفق، غیرصفر = خطا
Permission / chmod	دسترسی / تغییر دسترسی	rwx برای صاحب/گروه/دیگران
sudo	—	اجرا با هویت ادمین
Shebang	—	`#!.` خط اول اسکریپت: با چی اجرا شوم
Script	اسکریپت	فایل متنیِ قابل‌اجرای دستورات
Alias	نام مستعار	میان‌بر شخصی برای دستور بلند
REPL	محیط تعاملی	مثل `node` خالی — بنویس و فوری جواب بگیر

۱۳.۴ — ده ارور کلاسیک و ترجمه‌شان

ارور	یعنی چی	علاج
`command not found`	در PATH نبود	نصب؟ مسیر به PATH؟ (`which`)
`No such file or directory`	مسیر اشتباه	`pwd` و `ls` — کجام؟ چی اونجاست؟
`Permission denied`	اجازه نداری	`chmod` یا `sudo` (فقط واقعاً لازم)
`Is a directory`	فایل می‌خواستی، پوشه دادی	مسیر رو کامل کن
اسکریپت اجرا نمی‌شه با `.`	بزن `./script.sh`	نقطه‌اسلش = «همین‌جا»
`EADDRINUSE :3000`	پورت اشغاله	netstat → taskkill (فصل ۱۰!)
`rm: cannot remove...`	دسترسی یا r- لازم	`rm -r` ؛ محافظت‌شده؟ sudo
فلگ جواب نداد	شاید جای اشتباهی است	`--help` — ترتیب فلگ‌ها مهمه
خروجی لاگ خالی بود	خطاها در stderr بود	`2>&1` (فصل ۶)
متن فارسی به‌هم‌ریخته در ترمینال	فونت/یونیکد	Windows Terminal یا فونت مناسب

۱۳.۵ — نقشه راه پس از این دوره



توصیه‌های پایانی:

1. **قانون ۳۰ روزه:** یک ماه هر عملیات فایلی رو اول از ترمینال بزن (کپی، ساخت، جستجو). بعد از ۳۰ روز دست‌هات خودشون می‌رن سمت ترمینال — مثل رانندگی.
2. **هر چیز تکراری → اسکریپت.** دفعه سومی که یک کار رو دستی کردی، اسکریپتشو بنویس (فصل ۱۲). کلیدهای میان‌بر شخصی‌ات = مزیت واقعی تو در تیم.
3. **داکیومنت رسمی دوست:** `man bash` و gnu.org/software/bash مرجع کامل Bash؛ و برای Windows Terminal: `learn.microsoft.com` . این دوره روش فکر کردن رو داد؛ جزئیات بی‌انتها اونجاست.

مسیر رشد مهندسی: تسلط بر لینوکس و خط فرمان (CLI) پایه و اساس دنیای DevOps، مدیریت سرور، Docker، کانتینرسازی و پایپ‌لاین‌های CI/CD است. با تمرین و استفاده روزمره از مفاهیم این دوره، گام بلندی در حرفه‌ای‌تر شدن برداشته‌اید. 🚀

موفق باشید — ترمینال اکنون ابزار قدرتمند شماست!